



In the name of God

University of Tehran
Faculty of Electrical and Computer Engineering

Deep Generative Models

Homework 4 - Mohammad Taha Majlesi - 810101504

Name and Surname	Mohammad Taha Majlesi Kopaei
Student ID	810101504
Submission Date	2025/01/03

Contents

1	Part One: PaliGemma Model	4
1.1	Question 1	4
1.1.1	Answer	4
1.2	Question 2	5
1.2.1	Answer	5
2	Part Two: SigLIP Image Encoder	9
2.1	Question 1	9
2.1.1	Answer	9
2.2	Question 2	10
2.2.1	1. Replacing the Softmax-based Loss Function with a Pairwise Sigmoid Loss Function	10
2.2.2	2. Pairwise Sigmoid Loss Function	10
2.2.3	3. Advantages of Using the Sigmoid Loss Function in SigLIP	11
2.2.4	4. Practical Results and Impact on Training Speed	11
2.2.5	5. Final Conclusion	12
3	Pre-training	13
3.1	Question 1	13
3.1.1	Overall Review of the Stages (Overview)	13
3.2	Question 2	14
3.2.1	1. Prefix Tokens	14
3.2.2	2. Suffix Tokens	15
3.2.3	3. Why Distinguish Between Prefix and Suffix in PaliGemma	15
3.2.4	4. Practical Benefits	15
3.3	Question 3	16
3.3.1	1. Use of a Powerful Pre-trained Image Encoder	16
3.3.2	2. Gentle "Warm-up" Design for the Learning Rate of the Visual Part	16
3.3.3	3. Keeping the Connector between the Image Encoder and the Language Model Simple	16
3.3.4	4. Masking Structure of Prefix-LM Type	17
3.3.5	5. Not Completely Freezing the Language Part	17
3.3.6	6. Preventing "Training Everything at Once" (Gradual in Stage Design)	17
4	Part Four: Transfer Learning	18
4.1	PEFT: Parameter-Efficient Fine-Tuning	18
4.1.1	Reasons for Using PEFT	18
4.1.2	Principles of PEFT	18
4.1.3	Famous PEFT Methods	19
4.1.4	Advantages of PEFT	19
4.1.5	Disadvantages of PEFT	19
4.1.6	Practical Example	19
4.2	Question 1	20
4.2.1	1. LoRA (Low-Rank Adaptation)	20
4.2.2	2. QLoRA (Quantized LoRA)	21
4.2.3	3. LoRA Rank	21
4.2.4	4. NF4 Datatype	22
4.2.5	Target Modules in the Question (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, and down_proj)	22
5	CoPrompt	25
5.1	1. General Idea of CoPrompt: Maintaining Cohesion between the Pre-trained Model and the Fine-tuned Model	25
5.2	2. Introducing Perturbation in the Inputs for Better Training (Perturbed Inputs)	25
5.2.1	2.1. Text	25
5.2.2	2.2. Image	25
5.3	3. Combining Two Important Methods: Prompts + Adapters	26
5.3.1	3.1. Prompts	26

5.3.2	3.2. Adapters	26
5.3.3	3.3. Advantage of Combination	26
5.4	4. Why This Prevents Catastrophic Forgetting?	26
5.5	5. Achievements and Practical Results	26
5.6	Short Summary: CoPrompt	26
6	Part Five: Implementation of the Models	27
6.1	Report and Analysis of the leonardPKU/clevr_cogen_a_train Dataset	27
6.2	Content Analysis and Model Task	28
6.3	Connection with the Executed Code	28
6.4	Samples from the Dataset	28
6.5	Introduction to the paligemma-3b-mix-224 Model	29
6.5.1	1. Model Structure and Architecture	30
6.5.2	2. Main Specifications and Naming	30
6.5.3	3. Capabilities and Applications	30
6.6	Environment Setup and Dataset Loading	31
6.7	Model and LoRA Configuration	32
6.7.1	1. Loading the Processor	32
6.7.2	2. Loading the Model with Quantization	32
6.7.3	3. Configuring and Applying LoRA (Low-Rank Adaptation)	33
6.7.4	Explanation of the <code>preprocess_function</code>	34
6.8	Trainer Setup and Fine-Tuning	35
6.8.1	1. Defining Training Arguments (TrainingArguments)	35
6.8.2	2. Initializing the Trainer Class	36
6.8.3	3. Training Execution	36
6.9	Full Evaluation and Saving Results	36
6.9.1	1. Preparing and Loading Tools	37
6.9.2	2. Full Evaluation Loop	38
6.9.3	3. Quantitative Analysis with ROUGE and Saving Results	38
6.10	Evaluation Metrics	38
6.10.1	Quantitative Evaluation Metric: ROUGE Score	38
6.10.2	Qualitative Evaluation Metric (Saving Results)	39
6.11	Model Fine-Tuning Results	39
6.12	Report on the Progress of the Model's Fine-Tuning Process	40
6.12.1	1. Analysis of Training Strategy and Data Resources	40
6.12.2	2. Loss Trend and the Need for More Epochs	40
6.12.3	3. Saving and Public Access	40
6.13	Final Conclusion and Model Status	41
6.13.1	Best Performance (Best Model)	41
6.13.2	Severe Fluctuations in Training Loss	41
6.13.3	No Overfitting	41
6.14	Results	41
6.15	Analysis of Quantitative Results	42
6.15.1	Conclusion from Quantitative Analysis	42
6.15.2	Qualitative Analysis of Results	42
6.15.3	Root Cause of Performance Drop	43
6.15.4	Main Discrepancy: The Contradiction between ROUGE and Accuracy	44
6.15.5	Qualitative Analysis of Results	44
6.16	Final Summary of PaliGemma's Performance After Fine-Tuning	44
6.16.1	1. Success in the Main Goal: Increased Accuracy	45
6.16.2	2. Challenge in Formatting and Drop in ROUGE	45
6.16.3	3. Final Conclusion	45

7	Part Six: Flow Matching	46
7.1	Question 2: Flow Matching	46
7.1.1	Analysis of the Flow Matching Method	46
7.1.2	First Subsection: The Equivalence Relation of the Loss Functions	47
7.1.3	Second Subsection: Proof of the Conditional Vector Field Relation (Conditional Vector Field)	48
7.1.4	Third Subsection: The Optimal Transport (OT) Problem	49

1 Part One: PaliGemma Model

The PaliGemma model is an advanced model in the field of Vision-Language Models (VLMs) designed with the goal of combining and integrating visual and textual data to perform complex tasks such as responding to visual questions (VQA) and describing images. This model, leveraging its scalable architecture, is capable of processing and simultaneously understanding visual and linguistic information, making it a powerful tool in various artificial intelligence applications.

The architecture of the PaliGemma model consists of two main components: the SigLIP image encoder and a language decoder. The SigLIP encoder uses advanced techniques such as sigmoid functions for pre-training to extract key features from images and transfer them to a shared embedding space that is aligned with textual data. This allows for effective interaction between visual and linguistic data, enabling the model to respond to image-related questions in a coordinated and precise manner. On the other hand, the language decoder, using a transformer architecture, has the ability to generate relevant and accurate text based on the features extracted from the images.

One of the prominent features of the PaliGemma model is its Parameter-Efficient Fine-Tuning (PEFT), which uses methods like LoRA and QLoRA for optimization and reducing computational requirements. These methods allow the model to be quickly and efficiently fine-tuned for specific tasks, such as visual question answering, with minimal changes to its parameters. Additionally, PaliGemma has been able to optimize the process of generating new data and enhance its generative capabilities by using Flow Matching techniques, which improves the quality and accuracy of the model's linguistic responses.

Ultimately, the PaliGemma model, by combining an advanced architecture, optimization techniques, and strong generative capabilities, has established its position as one of the leading models in the vision-language domain. This model, by providing accurate and relevant answers to visual questions, has enabled widespread use in areas such as intelligent assistants, content management systems, and medical image analysis. Furthermore, the scalability of PaliGemma makes it very suitable for future developments in the field of artificial intelligence and multimedia learning.

1.1 Question 1

Explain the traditional visual or textual subsections. Also, describe Vision-Language Models (VLMs) and their differences from other models.

1.1.1 Answer

1. Traditional Visual Models Traditional visual models mainly focus on processing and analyzing visual data such as images and videos. These models utilize various architectures, each designed for specific tasks.

- **A) Convolutional Neural Networks (CNNs):**
 - **Architecture:** CNNs include Convolutional Layers, Activation Layers, and Pooling Layers.
 - **Function:** These networks extract local features from images and gradually transform them into higher-level features.
 - **Applications:** Object detection, image classification, face recognition, and video analysis.
- **B) Vision Transformers (ViTs):**
 - **Architecture:** Unlike CNNs that use local filters, ViTs leverage the Attention mechanism to model long-range dependencies in images.
 - **Function:** They divide images into small patches and embed each patch into a vector. Then, using transformers, they process these vectors.
 - **Applications:** Image classification, object detection, and other computer vision tasks.

2. Traditional Textual Models Traditional textual models focus on processing linguistic data such as written texts. These models use different architectures, each designed for specific tasks.

- **A) RNN-based Models (Recurrent Neural Networks):**
 - **Architecture:** Includes Recurrent Layers that process data sequentially.

- **Function:** Capable of modeling temporal dependencies in textual data.
- **Applications:** Machine translation, next-word prediction, and sentiment analysis.

- **B) Transformer-Based Models:**

- **Architecture:** Uses the Attention Mechanism to process sequences simultaneously, without the need for sequential processing.
- **Function:** Ability to understand complex relationships between words in a sentence.
- **Applications:** Machine translation, text generation, question answering, and semantic analysis.

3. Vision-Language Models (VLMs) and Their Differences from Traditional Models Vision-Language Models (VLMs) are a combination of visual and textual data processing that allows models to simultaneously analyze and generate content in both domains. These models, using hybrid architectures, have the ability to understand and interact between images and texts.

Differences with Traditional Models:

- **Data Integration:** Unlike traditional models that focus on only one type of data (visual or textual), VLMs process and interact with both visual and textual data.
- **Shared Embedding Space:** VLMs typically map visual and textual data into a shared embedding space, which allows for interaction and combination of information from both domains.
- **Multimedia Applications:** These models are capable of performing tasks such as visual question answering (VQA), image captioning, and generating multimedia content that requires a combined understanding of image and text.

Famous VLM Models:

1. **DALL-E:** A generative model that has the ability to create new images based on textual descriptions.
2. **Imagen:** A generative model with an emphasis on higher quality image generation.
3. **PaliGemma:** A versatile and scalable model using advanced techniques like PEFT and Flow Matching.

1.2 Question 2

Explain the different approaches in Vision-Language Models (VLMs). One of these approaches is the end-to-end approach, and another is the modular approach. Explain models like DALL-E and PaliGemma with these two approaches and compare their advantages and disadvantages in general.

1.2.1 Answer

1. Introduction to End-to-End and Modular Approaches in Vision-Language Models In the field of Vision-Language Models (VLMs), there are two main approaches for designing and implementing models: the End-to-End approach and the Modular approach. Each of these approaches has its own specific features, advantages, and disadvantages, which will be discussed in detail below.

A) End-to-End Approach

- **Definition:** In the End-to-End approach, all components of the model, from processing input data (textual and visual) to generating output (e.g., image or text), are trained together in a single, unified architecture.
- **Features:**
 - **Complete Integration:** All parts of the model are trained simultaneously and as a whole, without the need for separate interactions between modules.
 - **Simplicity of Implementation:** The design and implementation of the model are simpler as there is no need to manage interactions between different parts of the model.

- **Simultaneous Learning:** The model is able to learn from both textual and visual data simultaneously to improve performance.
- **Advantages:**
 - **High Performance in Combined Tasks:** The ability to learn complex relationships between textual and visual data due to the joint training of all parts.
 - **Creativity and Innovation:** The ability to generate creative and diverse outputs based on the combination of textual and visual data.
 - **Stability in Training:** The model may have better generalization in learning due to the simultaneous training of all parts.
- **Disadvantages:**
 - **High Computational Requirements:** Training End-to-End models usually requires significant computational resources.
 - **Less Flexibility:** Changing or improving a specific part of the model requires retraining the entire model.
 - **Generalization Challenges:** It may not perform well when faced with new data or different tasks.

B) Modular Approach

- **Definition:** In the modular approach, the model is divided into several independent parts, each performing a specific task and interacting with each other through defined interfaces.
- **Features:**
 - **Task Separation:** Each module is designed and trained independently to perform specific tasks.
 - **High Flexibility:** The ability to change or improve each part independently without the need to retrain the entire model.
 - **Better Resource Management:** Computational resources can be allocated more optimally, as each module can be optimized separately.
- **Advantages:**
 - **High Flexibility:** The ability to change, improve, or replace each part independently without affecting other parts.
 - **Optimal Resource Management:** The ability to optimize and allocate resources to each module based on its specific needs.
 - **Scalability:** The possibility of adding new modules for new tasks without directly affecting other parts.
- **Disadvantages:**
 - **Implementation Complexity:** The need to design specific interfaces and manage interactions between modules can be complicated.
 - **Coordination between Modules:** Ensuring optimal coordination and interaction between different parts of the model can be challenging.
 - **Lower Efficiency in Some Cases:** The lack of complete integration may reduce the model’s efficiency in some specific tasks.

2. Comparison of DALL-E and PaliGemma Models with End-to-End and Modular Approaches

A) DALL-E Model (End-to-End Approach)

- **Definition:** DALL-E is a generative vision-language model developed by OpenAI that has the ability to generate new images based on textual descriptions.
- **Approach:** DALL-E uses an End-to-End approach, where all parts of the model, from text input to image output, are trained as a single unit.
- **Features:**
 - **Transformer Architecture:** Uses a transformer architecture to combine textual and visual data.
 - **Generative Output:** The ability to produce creative and unique images based on textual descriptions.
 - **Extensive Pre-training:** Trained on a large dataset of images and textual descriptions.
- **Advantages:**
 - **High Creativity:** The ability to generate innovative and diverse images based on textual descriptions.
 - **Text and Image Integration:** The ability to accurately understand and combine textual and visual data.
 - **Stability in Training:** The model has the ability to learn complex relationships between text and image due to the simultaneous training of all parts.
- **Disadvantages:**
 - **High Computational Requirements:** Training and inference of the model require high computational resources.
 - **Low Flexibility:** Changing or improving a specific part of the model requires retraining the entire model.
 - **Potential for Incorrect Generation:** Sometimes the generated images may not exactly match the textual descriptions.

B) PaliGemma Model (Modular Approach)

- **Definition:** PaliGemma is a versatile and scalable vision-language model that is designed using advanced techniques such as PEFT (Parameter-Efficient Fine-Tuning) and Flow Matching.
- **Approach:** PaliGemma uses a modular approach, where the model is divided into independent parts such as the SigLIP image encoder and the language decoder, each of which is optimized separately.
- **Features:**
 - **Scalable Architecture:** The ability to increase the size and parameters of the model without reducing its efficiency, which provides the possibility of generalization and performance improvement.
 - **Fine-Tuning with PEFT:** Uses techniques like LoRA and QLoRA for fine-tuning the model with less memory and computation.
 - **Flow Matching:** Uses advanced generative techniques to improve the quality and efficiency of generating multimedia data.
- **Advantages:**
 - **High Scalability:** The ability to develop and tune the model for specific tasks with fewer resource consumption.
 - **Optimal Efficiency:** The use of PEFT techniques reduces computational and memory requirements, which allows the use of large models in limited environments.

- **Strong Generative Ability:** Flow Matching improves the quality and efficiency of generating generative data.

- **Disadvantages:**

- **High Flexibility:** The ability to change or improve each part independently without the need to retrain the entire model.
- **Implementation Complexity:** The need to design and manage specific interfaces between modules can be complicated.
- **Coordination between Modules:** Ensuring effective coordination and interaction between different parts of the model can be challenging.
- **Lack of Integration:** In some specific tasks, it may have lower efficiency than End-to-End models.

Features	Imagen	DALL-E
Approach	End-to-End	End-to-End
Architecture	More advanced transformer	Transformer
Image Quality	Very high	Good
Fine-Tuning	Limited	Limited
Stability	High	Medium
Computational Requirements	Very high	High
Flow Matching	Not available	Not available
Advantages	High quality, higher stability	Creativity, text and image integration
Disadvantages	More complexity, need for more data	Need for many resources, incorrect genera

Table 1: Comparison of Imagen and DALL-E

3. General Comparison between Imagen and DALL-E Models

4. Conclusion Vision-language models like DALL-E and Imagen, using an End-to-End approach, have been able to provide significant generative capabilities, including the creation of creative and high-quality images based on textual descriptions. These models, with complete integration between the textual and visual parts, have the ability to learn complex relationships between these two types of data. However, high computational requirements and limited flexibility are among the main disadvantages of these approaches.

On the other hand, the PaliGemma model, using a modular approach and leveraging advanced techniques such as PEFT and Flow Matching, has been able to establish a balance between efficiency, scalability, and generative quality. This model, with the ability to fine-tune and optimize different parts independently, offers higher flexibility and can be used with fewer computational resources. However, the complexity of implementation and the need for precise coordination between modules are among the main challenges of this approach.

Final Conclusion: The choice of the appropriate approach between End-to-End and modular depends on the needs, available resources, and specific features of the project. If the focus is on producing creative and high-quality images with sufficient computational resources, models like DALL-E and Imagen with an End-to-End approach can be suitable options. But if there is a need for more flexibility, fine-tuning, and resource optimization, models like PaliGemma with a modular approach can be a better choice.

2 Part Two: SigLIP Image Encoder

In the following, we provide the answer text with full details, in a fluent and understandable Persian language:

2.1 Question 1

”How is the model architecture? How does the process of training the Joint Embedding Space work? And what Loss Function is used to train the SigLIP model?”

2.1.1 Answer

The SigLIP (Sign Language Image Processing) model, which is used in the PaliGemma architecture, is essentially a modified version of the well-known CLIP (Contrastive Language-Image Pretraining) model. Both models arrange images and textual data in a common space so that similar image-text pairs can be easily compared or retrieved. The main focus of SigLIP is on recognizing and processing sign language, but its methodological basis is the same as CLIP.

In the following, we will have a deep dive into the architecture, training mechanism, and loss function in SigLIP.

1. Architecture

- **Image Encoder:** In SigLIP, the image encoder is usually a Convolutional Neural Network (CNN) or a Vision Transformer (ViT).
 - **CNNs** are conventional architectures for image processing and, with convolutional layers, gradually extract higher-level features from raw pixels.
 - **ViT**, which is a newer approach, considers the image as a collection of patches (similar to tokens in natural language processing) and uses the transformer mechanism to examine the relationship between these image parts.

This encoder converts the input images into high-dimensional, content-rich vectors that represent the semantic content of the image.

- **Text Encoder:** The text encoder is usually based on a transformer model (like BERT or GPT). This model processes linguistic descriptions (or textual labels) and maps them to a high-dimensional vector space; a space that is dimensionally consistent with the image encoder. This process causes textual descriptions (e.g., descriptions of sign language or labels) to be converted into vectors that can be compared with the image vectors.
- **Joint Embedding Space:** The key point in CLIP and SigLIP is this common space; where both the image encoder and the text encoder map their inputs (image and text) to the same space. In this space, images that are associated with a specific text description should be close to each other, and unrelated pairs should be far apart. This feature allows for tasks such as Cross-Modal Retrieval, for example, finding images based on text or finding texts based on an image.

2. Training the Joint Embedding Space

- **Data Pairs:** The model is trained on large datasets that include image-text pairs. In SigLIP, these pairs are related to sign language images and their textual descriptions (or interpretations).
- **Training Objective:** During training, both the image encoder and the text encoder are optimized simultaneously to ensure that the embeddings (vectors) corresponding to a real image-text pair are aligned and close to each other. In other words, similar image-text pairs should have close vectors in the common space, and dissimilar pairs should be far from each other.

3. Loss Function In SigLIP, a Contrastive Loss function is used, which is essentially similar to the loss function of the CLIP model.

How the Contrastive Loss Function Works: In CLIP and SigLIP models, a symmetric Cross-Entropy is usually used in a comparative form, which achieves two main goals:

- **Positive Pairs:** The embeddings of the image and text that are related to each other should be brought closer in the embedding space (reducing the cosine distance or increasing the cosine similarity).
- **Negative Pairs:** The embeddings of the image and text that are not related to each other should be pushed away from each other in this space (increasing the cosine distance or decreasing the cosine similarity).

Calculation Method (Mathematical Example): Functions such as Triplet Loss or NT-Xent (Normalized Temperature-scaled Cross-Entropy) are commonly used, which compare the similarity of each image-text pair with all other pairs in a batch. Its general form is as follows:

$$L = -\log \frac{\exp(\text{sim}(I, T)/\tau)}{\sum_j \exp(\text{sim}(I, T_j)/\tau) + \sum_k \exp(\text{sim}(I_k, T)/\tau)}$$

where:

- $\text{sim}(I, T)$ represents the cosine similarity between the image vector I and the text vector T .
- τ is a parameter for adjusting the scale (temperature); the smaller τ is, the more the loss function tends to make the similarity differences sharper.
- In the denominator of the fraction, the sum of the similarities of all other image-text pairs in a batch is placed. This causes the model to reduce the distance between positive pairs and increase the distance between negative pairs.

With the help of comparative learning, the model learns to maximize the similarity between real (positive) pairs and minimize it between unrelated (negative) pairs; as a result, a well-aligned common space is created.

2.2 Question 2

"In the SigLIP training process, an idea has been used that leads to faster training compared to CLIP. What is this idea and what is its advantage?"

2.2.1 1. Replacing the Softmax-based Loss Function with a Pairwise Sigmoid Loss Function

The CLIP (Contrastive Language-Image Pre-Training) model uses a contrastive loss function based on Softmax, which requires creating and maintaining a similarity matrix ($N \times N$) for all images and texts in a batch. As the batch size increases, this approach becomes very costly in terms of memory and inter-device communication (All-Gather).

In contrast, SigLIP (Sigmoid Language-Image Pre-training) utilizes a pairwise sigmoid loss function and solves a binary classification problem (positive or negative) for each image-text pair independently. This eliminates the need for global normalization across the entire batch and removes the ($N \times N$) matrix.

2.2.2 2. Pairwise Sigmoid Loss Function

In this method, the model tries to determine for each pair of image (I) and text (T) whether this pair is a "correct match" (label = 1) or "unrelated" (label = 0). Therefore: (e.g., based on cosine similarity or dot product)

1. Sigmoid Function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

whose output is in the range (0,1) and indicates the probability of the pair being "positive".

2. Loss Function (Binary Cross-Entropy):

$$\text{loss} = \text{BCE}(\sigma(\text{sim}(I, T)), \text{label})$$

In detail, the pairwise sigmoid loss function is written as follows:

$$\text{loss} = -[y \cdot \log(\sigma(\text{sim}(I, T))) + (1 - y) \cdot \log(1 - \sigma(\text{sim}(I, T)))]$$

where:

- y is the label of the pair (0 or 1).
- $\sigma(\cdot)$ is the sigmoid function.

2.2.3 3. Advantages of Using the Sigmoid Loss Function in SigLIP

1. Elimination of Global Normalization (Softmax) over the entire batch: In CLIP, for each image, the probability of belonging to each text present in the batch must be calculated (and vice versa), which requires a similarity matrix ($N \times N$).

- In SigLIP, each pair is examined independently, and there is no need for a large matrix for normalization.

2. Significant Reduction in Memory Consumption: By eliminating the ($N \times N$) matrix, the memory required for processing image-text similarities is greatly reduced. This issue is particularly vital for large batch sizes (e.g., 16K, 32K, 64K, and larger).

3. Reduction of Communication Overhead:

- In the Softmax approach, all image and text embeddings must be exchanged between computational devices (All-Gather) to obtain the complete matrix.
- In the pairwise sigmoid method, instead of all-for-all (All-Gather), embeddings can be exchanged only in a chunked manner or only with neighboring devices, and enough negative pairs can be obtained for the loss calculation.

4. Easier and More Stable Implementation:

- The calculation of $\log \sum \exp$ in Softmax requires numerical stabilization (e.g., subtracting the maximum logit).
- In sigmoid, each pair produces only one logistic value, which is directly placed in the Binary Cross-Entropy formula, and its implementation is simpler and more stable.

2.2.4 4. Practical Results and Impact on Training Speed

- **Increased Scalability:** By eliminating the need for the ($N \times N$) matrix, the batch size can now be made very large (even up to hundreds of thousands or a million) without the device's memory running out quickly.
- **Improved Speed and Throughput:** Since both communication costs are reduced and there is no need for multiple numerical passes, SigLIP can achieve the same accuracy or even higher accuracy in less time.
- **Very Large Models on Limited Hardware:** The paper mentions that a batch size of 32K can be run on a few TPU/GPUs without trouble. In CLIP, such a size usually requires a much larger cluster.

2.2.5 5. Final Conclusion

The key idea in SigLIP is that instead of relying on a function that needs to be normalized over all texts for each image (and vice versa) (the same Softmax in CLIP), it examines each image-text pair independently and uses a pairwise sigmoid function to determine if the pair is positive or negative. This:

- Drastically reduces memory consumption.
- Reduces communication between devices.
- Makes the process scalable for very large batch sizes.
- Makes implementation and computations easier and faster.

Final Formula for the Pairwise Sigmoid Loss Function in the form of Binary Cross-Entropy:

$$\text{loss}_{\text{sigmoid}}(I, T) = -[y \log(\sigma(\text{sim}(I, T))) + (1 - y) \log(1 - \sigma(\text{sim}(I, T)))]$$

Where:

- I and T are the embeddings of the image and text.
- $\text{sim}(I, T)$ is a similarity function (e.g., dot product or cosine).
- σ is the sigmoid function.
- $y \in \{0, 1\}$ is the label indicating whether the pair is positive (1) or negative (0).

3 Pre-training

3.1 Question 1

After the image encoder and text decoder are trained separately on uni-modal data, in the next stage, these two models are combined or placed in the overall architecture of the model and are trained again in two more stages (pre-train). How are these two training stages performed, and what is the necessity of each?

3.1.1 Overall Review of the Stages (Overview)

1. Stage 0: Unimodal Pre-Training

- **Summary:** The Vision Encoder and the Language Model are each trained separately on vast data related to their own domain. For example, SigLIP is trained only with image-text pairs for learning visual representation in a contrastive manner, and Gemma learns a self-regressive language model on large text datasets.
- **Objective:** To ensure that each of these parts (visual or linguistic) becomes powerful on its own and provides strong performance in its unimodal task. As a result, in the next stage when these two are placed together, their initial foundation is good, and there is no need to learn everything from scratch.

2. Stage 1: Multimodal Pre-Training

- **Summary:** The combination of SigLIP (as the image encoder) and Gemma-2B (as the language decoder) in a single model. From now on, the entire model is trained with various multimodal tasks. In this stage, the input resolution is usually 224x224 pixels.
- **Important Details:**
 - **Without freezing the entire parts:** Unlike the procedure where sometimes the image encoder or the language model is frozen so that their pre-trained parameters are not disturbed, in PaliGemma, all parameters (both visual and linguistic) are updatable. This allows the model to learn to integrate image and text in a deeper way.
 - **Gentle Warm-up for the image encoder's learning rate:** If we set a large learning rate for the visual part from the very beginning, it is possible that irrelevant gradients from the language side will severely damage the quality of the image encoder. To prevent this, the learning rate of the image encoder is initially set very low and is gradually increased. As a result, mismatched signals in the initial stages do less damage to the encoder, and gradually the two-way model finds a better convergence.
 - **Use of several different tasks:** In multimodal datasets, in addition to captioning and visual question answering (VQA), tasks such as object detection, segmentation of image parts, text recognition in images (OCR), etc., are also included. This diversity helps the model to acquire a wide range of skills in "understanding image and language".
 - **Architectural Design:**
 - * First, the output of the image encoder (image tokens) passes through a linear projection layer to have the same dimensions as the language tokens (Gemma's vocabulary).
 - * Then, the image tokens + BOS + prefix tokens + SEP + suffix tokens + EOS are sent to the language model (as a sequence).
 - * The attention mechanism is designed as prefix-LM; meaning that the image tokens and prefix tokens can see each other (bidirectionally), but the suffix tokens only have access to their own past (auto-regressive). This pattern is essential for the model to be generative, meaning the model can predict the final phrase.
- **Objective:** The model can now process image and text simultaneously and be trained on a huge collection of multimodal tasks; therefore, the skill of correlation between "visual vectors" and "linguistic interpretation" is formed. At the end of Stage 1, the model can both read an image from start to finish (by the SigLIP encoder) and produce a response in the form of language (with Gemma).

3. Stage 2: Resolution Increase

- **Summary:** The same trained model from Stage 1 is now continued for a limited period with a higher image resolution (usually 448x448 or 896x896).
- **Important Details:**
 - **Resolution-sensitive tasks:** In this stage, more training data related to reading fine text (OCR), charts and diagrams, as well as tasks related to small objects or detecting/segmenting fine objects are given higher priority. Because these tasks really need more pixels, and at 224x224, sufficient information may not be available.
 - **Relatively short training:** Unlike Stage 1, which may see several hundred million or even a billion samples, this stage only sees tens of millions of new samples (but they are more expensive because they have higher resolution). The idea is that the model has learned its general "multimodal" knowledge in Stage 1 and now just learns how to deal with more image details.
 - **Benefit of increasing image tokens:** When the image becomes larger, the number of image tokens (output of the ViT encoder) also increases. This means the model has more capacity to include more precise positional information in the token code. The manifestation of this issue is seen in OCR, segmentation, etc. tasks.
- **Objective:** Improving the model in fine-grained tasks. In practice, the paper shows that the model's performance on reading small texts, UI designs, challenging issues in VQA (e.g., precise information like numbers on a chart or object details) is improved by passing through Stage 2.

4. Stage 3: Final Transfer

- **Summary:** The result of all stages (encoder + decoder) is now available as a base multimodal model. But it may still be necessary to "fine-tune" it for a specific task or application.
- **Why Fine-tuning is needed:** Sometimes we want to solve specific tasks better; for example, Video QA, or a dataset of a specific field like medical images. Or we want to instruction-tune the model so that it can act in a conversational style.
- According to the paper's results, this same base model can be tuned on a specialized dataset (e.g., COCO Caption, InfographicQA, WidgetCap, etc.) in a short time and achieve high performance.

3.2 Question 2

In the PaliGemma architecture, the input sequence to the Gemma language model is divided into two main parts: "prefix tokens" and "suffix tokens". This division is rooted in how "input data" interacts with the "auto-regressive text generation system". In the following, without giving a specific example, we explain the role and necessity of each in a conceptual and technical way.

3.2.1 1. Prefix Tokens

A) Task and Input Condition Identification Prefix tokens are generally for explaining the task that the model should perform. This could be "visual question answering", "captioning", "object detection", or any other task. Since in PaliGemma, a large language model (LLM) is responsible for generating text, it needs to know what is being asked of it. So, in the prefix section, keywords or phrases related to the type of task, target language, or other explanations are provided.

B) Bidirectional Attention to Image and Prefix Since the PaliGemma architecture uses the prefix-LM approach for the input stage, the prefix tokens can initially have bidirectional interaction with each other (and with the image tokens); meaning they look at each other and the visual data, and the attention layers see each of them. In this way, the model can better decide what aspects of the image or what style of response it should follow.

C) Differentiating Different Tasks When the model is trained on various multimodal datasets, these prefix tokens act as a "label" or "navigator" to show the model what kind of task it is currently performing. This prevents interference in training for different tasks and determines the model's orientation from the very beginning.

D) Need for Non-Autoregressive Prediction In the prefix-LM method, it is assumed that all prefix tokens and image tokens are data that we have "from before" and should not be predicted autoregressively. That is, these tokens are part of the "context" and the model is supposed to see all of them to construct the final response based on them. Therefore, the prefix tokens play a context-provider role.

3.2.2 2. Suffix Tokens

A) Step-by-Step (Autoregressive) Generation The most important feature of the suffix tokens is that the language model must "build" them step-by-step. This is a fundamental difference from the prefix section, which was predetermined and fixed. In the suffix section, the model can only look at the tokens it has generated so far (as well as the entire prefix and image), but it is not allowed to know the future of the response. This "autoregressive" structure is the core of generative language models.

B) Main Output of the Model Whatever is requested from the model (description, answer, a list of coordinates, translated text, etc.) is formed in this area. The model, based on the visual content and the prefix text, decides what the next token should be and writes it. In this way, the entire "response" is placed in the suffix sequence until it finally ends.

C) Lack of Bidirectional Vision Unlike the prefix, whose tokens can see each other bidirectionally, in the suffix section, the model is not allowed to look at "future" tokens; that is, a suffix token only knows about the prefix + image + suffix tokens that have been generated before. This prohibition of "looking at the future" is part of the main mechanism of the autoregressive language model and ensures that the text generation proceeds step by step.

D) End (EOS) and Padding (PAD) To stop the generation process, the end marker (EOS) is placed at the end of the suffix. This means the model says "the work is done". And if necessary, the sequence is filled with PAD tokens to a fixed length. Therefore, the suffix is the part where the language model is free to create the content of the response as much as it needs to reach a conclusion.

3.2.3 3. Why Distinguish Between Prefix and Suffix in PaliGemma

A) Compatibility with Multimodal Approaches The model first freely sees the image and prefix tokens, so it can understand all the environmental, instructional, and visual information at once. Then, in the form of a suffix, it reacts step by step and generates the output text. This structure is compatible with both the nature of image processing (which is given in advance) and the nature of "autoregressive language" in the output generation section.

B) Support for a Diverse Range of Functions When it is necessary for the language model to participate in different tasks (captioning, question answering, object detection, etc.), we simply change the instruction in the prefix section; but the suffix section remains the place for generating text that proceeds according to that instruction.

C) No Interference between Instruction and Response If the instruction (question or any format) and the response were mixed, the model would have to determine which part of the input "should be predicted" and which part is "just a fixed input". But by separating the prefix and suffix, this problem is solved. The prefix is always the input and is not predicted; the suffix is always the output and must be predicted step-by-step.

3.2.4 4. Practical Benefits

1. **Stable Convergence:** Since the masking structure in the response section is autoregressive and in the question/image section is bidirectional, the model is trained in a relatively simple and controllable way. Some of the errors and potential inconsistencies that occur in complex architectures are reduced.

2. **Using Prefix Knowledge for Better Decoding:** When a question or instruction is in the prefix, all suffix tokens can access it. As soon as each token is generated in the suffix, it benefits from the content of the prefix (including the concept of the task, language, special instructions, etc.). This flow of information causes the response to be consistent with the task's need.
3. **Extensibility and Adding New Tasks:** In the future, if we define a new task or structure, it is enough to add a new keyword or format in the prefix so that the model knows what the job is. The output will still be in the suffix. In this way, a highly flexible architecture is created.

3.3 Question 3

One of the common challenges in such structures is that the language model is fine-tuned on text tokens, and when it encounters visual data, its quality may drop. How has the PaliGemma model solved this problem?

In many multimodal approaches, when a large language model (LLM) is directly mixed with visual input, there is a possibility that the "linguistic knowledge" learned before will be weakened or even partially forgotten under the influence of visual signals and new gradients. This phenomenon is called Catastrophic Forgetting. In such a situation, the model may still be able to extract information from the image, but at the same time, its linguistic skills are lost, or its overall quality drops.

In the PaliGemma model (and similar methods), several simultaneous solutions are used to prevent the destruction or decline of the LLM's performance when receiving visual data:

3.3.1 1. Use of a Powerful Pre-trained Image Encoder

- **SigLIP:** This image encoder has been pre-trained on a huge dataset of image-text pairs using the contrastive learning method (in Stage 0).
- **Main Advantage:** When the language model receives high-quality and meaningful vectors instead of raw image pixels, it does not need to learn pixel decoding "from scratch". Therefore, the incoming visual signal is structured and stabilized and is less likely to cause sudden changes in the language model's parameters.

3.3.2 2. Gentle "Warm-up" Design for the Learning Rate of the Visual Part

- In the multimodal stage (Stage 1), if a very large learning rate (LR) is set for the image encoder from the very beginning, the language model, which is not yet coordinated with the visual signal, may send strange and bizarre gradients to the image encoder.
- **Solution:** Initially, the LR of the image encoder is set very low and is gradually increased to the desired amount.
- **Result:**
 1. In the initial stages, when the language model still lacks sufficient "adaptation" to the visual data, fewer changes are applied to the visual part (prevention of destruction).
 2. As soon as the language model and the multimodal signal become somewhat coherent, the LR of the image encoder is increased and can improve learning in line with complex tasks (like spatial relationships or fine details).

3.3.3 3. Keeping the Connector between the Image Encoder and the Language Model Simple

- **Linear Projection Layer:**
 - Many works, to integrate the image input into the language model, use a multi-layer MLP network or more complex mechanisms.
 - In PaliGemma, a simple linear layer is used to convert the output dimensions of the image encoder to the same dimensions as the language tokens.
- **Advantage:** The simplicity of this layer also leads to more stability in training. It also reduces the risk of "getting everything mixed up". The larger the intermediate layer, the more likely the language model will get "stuck" in it and an abnormal form of convergence or even a general drop will be created.

3.3.4 4. Masking Structure of Prefix-LM Type

- In traditional "Encoder-Decoder" architectures, some input tokens may also be predicted in an autoregressive manner, or the visual and textual parts may have a complex mask distribution.
- In PaliGemma, the Prefix-LM method is used:
 1. The image tokens + the question/instruction tokens (prefix) can be viewed bidirectionally. That is, these tokens see each other and "deliberate" together.
 2. But the response tokens (suffix) are generated in an autoregressive manner; each subsequent token is written based on the previous tokens.
- **Why it helps:**
 1. The image tokens from the very beginning can "know" what the question or text instruction is (prefix). So, the image vectors purposefully signal the need to pay attention to specific parts of the image.
 2. The language model is only in the response section (suffix) that has to generate output step-by-step; therefore, the "linguistic brain" does not undergo a strange processing of the entire input (including the visual part) in an autoregressive manner. As a result, the linguistic knowledge is less damaged.

3.3.5 5. Not Completely Freezing the Language Part

- Unlike some methods that say "to protect the language model, let's freeze it and only make adjustments on the visual part", this is not the case in PaliGemma.
- **But:** Since SigLIP is already powerful and also has a gentle warm-up, the gradients are guided in a way that the language model is not weakened.
- **Benefit:**
 1. The LLM can learn from the visual signal, for example, how to give a precise linguistic response to complex images (e.g., where to count, where to describe...).
 2. The language model preserves its abilities in semantic and linguistic understanding and even becomes richer (because now it also gains more knowledge from images).

3.3.6 6. Preventing "Training Everything at Once" (Gradual in Stage Design)

- The PaliGemma approach is completely staged:
 - Stage 0 (unimodal training) -> Stage 1 (a few hours or a few days, but a wide variety of image-text data) -> Stage 2 (high resolution, but shorter duration).
 - This itself prevents excessive pressure that might crush the language model with raw/massive visual inputs from the beginning.
- In the end, we have a model that not only does not drop but also performs well in various vision-language tasks.

4 Part Four: Transfer Learning

In this section, we first define PEFT:

4.1 PEFT: Parameter-Efficient Fine-Tuning

PEFT, short for Parameter-Efficient Fine-Tuning, is an innovative approach to fine-tuning large deep learning models. This method, instead of updating all the parameters of a large model, only changes a small subset of the parameters. The main goal of PEFT is to reduce the computational resources, memory required, and the cost of fine-tuning the model, without a significant drop in accuracy or performance.

4.1.1 Reasons for Using PEFT

1. **Size of Large Models:** Modern deep learning models (like BERT, GPT-3, or ViT) usually consist of billions of parameters. Fine-tuning these models completely (Full Fine-Tuning) requires a lot of memory and computation.
2. **High Cost and Time:** Full fine-tuning of large models for each task requires powerful hardware resources and a lot of time, which may not be practical for many users.
3. **Increasing the Usability of Models:** With PEFT, pre-trained models can be fine-tuned for specific applications with minor changes, without the need for retraining the entire model.

4.1.2 Principles of PEFT

Full finetuning vs PEFT

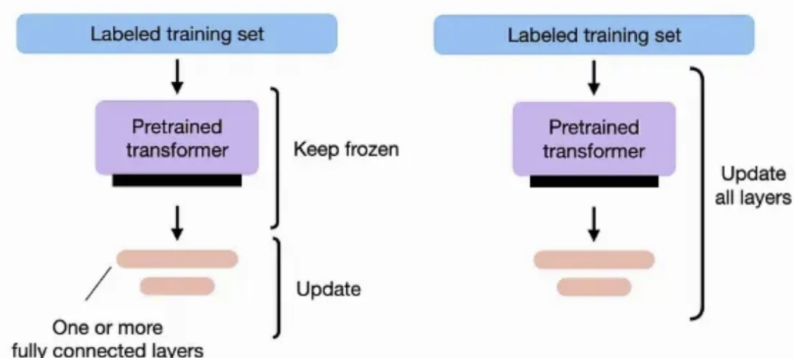


Figure 1: Full finetuning vs PEFT

In PEFT, only a small portion of the model's parameters are changed, or additional parameters are added to the model, while the bulk of the parameters remain fixed. This method usually includes the following steps:

1. **Keep the Main Model Fixed (Frozen Model):** The main parameters of the pre-trained model are kept fixed, and only a specific (usually small) part of them is changed.
2. **Add Extra Layers (Optional):** In some PEFT methods, new small layers or modules (like Adapters) are added to the model, which are responsible for learning the changes for a specific task.
3. **Selective Update:** Instead of changing all the parameters, only a specific part of the model (e.g., the last layers or special modules) is updated.

4.1.3 Famous PEFT Methods

1. **Adapters:** In this method, small layers are added between the main layers of the model. These layers are responsible for learning the changes related to a specific task. The main parameters of the model remain fixed, and only the Adapter parameters are updated.
2. **LoRA (Low-Rank Adaptation):** An effective method in PEFT that assumes the changes needed to fine-tune the model can be limited to a low-rank space. LoRA changes only a part of the model's parameters, usually by adding low-rank matrices to the model.
3. **Prefix-Tuning:** In this method, additional vectors (Prefixes) are added to the model's input, which guide the model for a specific task. These vectors are trained, but the model itself remains fixed.
4. **Prompt-Tuning:** Similar to Prefix-Tuning, but focuses only on the input settings (Prompts). The main model does not change at all.
5. **BitFit:** This method only fine-tunes the biases of the model and keeps all other parameters fixed.
6. **Diff Pruning:** This method assumes that the difference between the original model and the fine-tuned model can be stored sparsely.

4.1.4 Advantages of PEFT

1. **Reduction of Computational Resources:** Since only a small portion of the parameters are updated, the need for memory and computation is reduced.
2. **Faster Fine-Tuning:** Fine-tuning large models is time-consuming, but PEFT makes this process faster.
3. **Savings in Storage:** Instead of storing the entire fine-tuned model, only the small changes are stored.
4. **Increased Flexibility:** The main model can be used for several different tasks without general changes.
5. **Greater Access to Large Models:** PEFT makes it possible for users with limited resources to use large models.

4.1.5 Disadvantages of PEFT

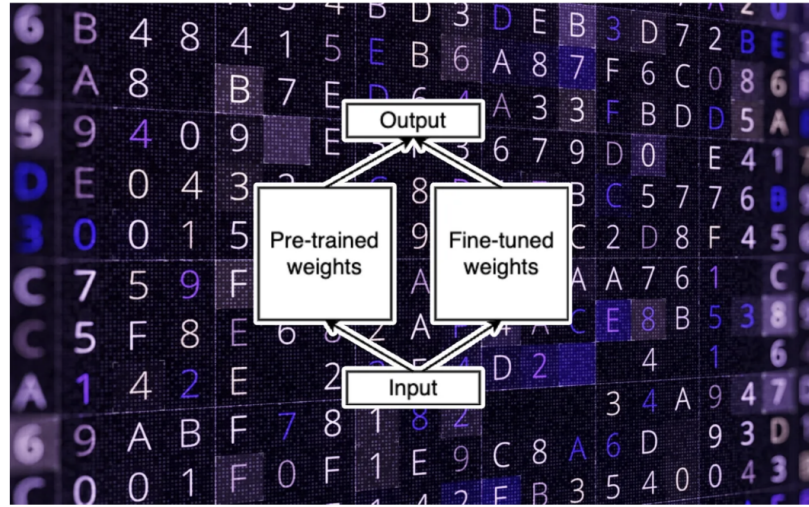
1. **Limitation in Complex Settings:** For complex tasks or data, its performance may not be as good as full fine-tuning.
2. **Need for Careful Design:** Choosing the parts of the model that should be updated requires deep knowledge of the model and the task.
3. **Lower Performance in Some Cases:** If extensive changes are needed in the model for a specific task, PEFT may not be as effective as Full Fine-Tuning.

4.1.6 Practical Example

Suppose you have a large model like GPT-3 and you want to fine-tune it for sentiment analysis. By using PEFT:

1. The main model remains fixed, and only the added layers or biases are updated.
2. You need much less memory and computation because only a small number of parameters are changed.
3. The final model will be lighter and faster, and you can easily use it for other tasks as well.

LoRA



LoRA approach. Source:[6]

Figure 2: LoRA approach. Source:[6]

4.2 Question 1

In this answer, we will explain each of the mentioned sections about LoRA, QLoRA, the Rank parameter in LoRA, and the NF4 datatype in more detail to clarify what happens in practice and what effect each one has.

4.2.1 1. LoRA (Low-Rank Adaptation)

1.1. Main Idea in LoRA When we want to fine-tune a large language model (LLM) or a vision-language model for a new task, we usually need to update all the model's parameters, which has a high memory and computational cost. In the LoRA method, we decide to keep the main weights of the model (e.g., the large matrices of the attention layer) fixed and instead, in the target sections (e.g., the q-proj, k-proj layers in self-attention), we add low-rank matrices that are the only ones updated during learning. These low-rank matrices are usually formed from dimensions $\text{fan_in} \times r$ and $r \times \text{fan_out}$, where r is the same Rank.

1.2. Why "Low-Rank"? The idea of "low-rank" is that the required change in the large model's weights is generally structured and has a lower effective dimension than the total dimensions of that layer. For example, if a layer has 65K parameters, the desired change can be implemented with a much smaller matrix (lower rank). In this way, the space in which the model can move relative to the "base" weights is more limited but sufficient to acquire new knowledge in the new task.

1.3. Detailed Advantages

- 1. Memory Savings:** In full fine-tuning, each model weight must have a gradient, momentum, and variance of the optimizer (e.g., in Adam), which creates a large volume. In LoRA, instead of all the weights, only the adapters with small dimensions + gradient + momentum + variance related to them are stored.
- 2. Few Parameters and Modular Structure:** After learning, there is no need to store the model completely. We can keep the same base weights + the small-volume adapters as a plug-in.
- 3. Speed:** Because the number of parameters being updated is small, backpropagation is performed faster, and less memory exchange occurs.

1.4. Limitations and Design Points

1. **Limited Capacity:** If the rank is too small or inappropriate modules are chosen for adding LoRA, the model may not be able to learn all the complexities of the task (underfitting). For this reason, sometimes it is necessary to increase the rank or apply LoRA to more modules to improve the quality.
2. **Design Requires Experience:** Depending on the model, sometimes LoRA is only applied to the q_proj and v_proj modules, sometimes all attention projections or even the feedforward are covered. This decision directly affects the performance and final volume.

4.2.2 2. QLoRA (Quantized LoRA)

2.1. Main Idea in QLoRA In QLoRA, we have the same low-rank LoRA adapters, but with the difference that we store the base weights of the model in 4-bit Quantized form. Therefore, the memory occupied by the main model is greatly reduced; but the LoRA adapters are still stored and updated with higher precision (e.g., FP16 or BF16).

2.2. Why 4-bit Quantization in the Base Model?

1. **Reduction of Memory and Hardware Cost:** For example, if you store a 10 billion parameter model in FP16, you need 20 gigabytes for just the weights. But in QLoRA, this can be reduced to a quarter (or even less).
2. **Combination with LoRA:** Although the weights are quantized, the main learning is on the low-rank adapters that have high precision and allow the quality to be preserved.

2.3. Limitations and Subtleties

- **Complexity of Quantization:** If the 4-bit implementation is not done correctly, the base model may suffer a significant drop in accuracy; especially if the normalization or numerical mapping method is wrong.
- **Remaining LoRA Limitations:** Ultimately, QLoRA is also exactly like LoRA dependent on the choice of rank, modules, etc. It just provides a greater advantage in reducing memory.

4.2.3 3. LoRA Rank

3.1. Definition of Rank When a layer, for example, has W , and we want to change it with LoRA, we add two low-rank matrices $A \in R^{d \times r}$ and $B \in R^{r \times d}$ (in summary, the parameter r is the same Rank). In practice, a weight of the form $W + \alpha \cdot AB$ is formed (where α is usually a small scaling factor).

3.2. High Rank vs. Low Rank

1. **High Rank:** The low-rank matrix AB covers a larger space; therefore, it can learn more diverse complexities of the new task. But it has more additional parameters (approximately $2 \times d \times r$) and requires more memory and time for updating.
2. **Low Rank:** It is lighter. It trains faster. But if the new task is very different or complex, the model with a low rank may not be able to create an accurate representation of it (drop in accuracy).

3.3. Finding the "Optimal Rank" A general principle is to use average numbers (like 8, 16, or 32), and experience has shown that it is sufficient for many language tasks. Sometimes it is necessary to experiment and evaluate on a validation dataset to choose the appropriate rank; a balance between quality and cost.

4.2.4 4. NF4 Datatype

4.1. Difference with simple int4 In the int4 method, the weights may be mapped linearly between min and max; or they are all stored with a fixed scale. In NF4 (Normalized Float 4-bit), a smarter approach is usually used:

- For each block of channels or each block of several tens of parameters, a scale factor is calculated to normalize that section.
- Then the numbers are stored in a 4-bit float range.

4.2. Advantage of NF4

- **Better Precision in Weight Distribution:** Because the numerical range of each block is adjusted separately, the quantization error for large or small ranges is reduced.
- **Preservation of Important Weight Features:** For example, if some weights are positive and some are negative, or different vectors have different ranges, NF4 works better than simple int4.
- **Bandwidth Reduction:** During the forward pass, reading the weights from memory is lighter, and computational resources are freed up more.

4.3. Application in QLoRA QLoRA uses NF4 to store the frozen weights of the model; that is, the base model is stored with NF4. Then, during training and inference, these weights are converted to a higher format (e.g., FP16) internally to perform calculations, but the storage and transfer in memory are with the 4-bit format.

4.2.5 Target Modules in the Question (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, and down_proj)

The target modules in the question (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, and down_proj) are main components of the architecture of transformer-based models (like GPT, BERT, or LLaMA). These modules are responsible for the calculations and transformations that are performed in the attention mechanism and the feed-forward layers. In the following, we explain each one completely.

1. Self-Attention Mechanism Modules The attention mechanism is a part of the model that calculates the relationships between the tokens of a sentence (or input tokens). These modules include the following:

- **q_proj (Query Projection):**
 - **Purpose:** Transforms the model’s inputs (embedded vectors or hidden states) into the query space.
 - **Role:** Determines what each token wants to pay attention to in other tokens.
 - **Mathematical Formula:** $Q = XW_q$, where X is the input and W_q is the query weight matrix.
- **k_proj (Key Projection):**
 - **Purpose:** Transforms the inputs into the key space.
 - **Role:** Indicates what information each token provides for other tokens to pay attention to.
 - **Mathematical Formula:** $K = XW_k$, where W_k is the key weight matrix.
- **v_proj (Value Projection):**
 - **Purpose:** Transforms the inputs into the value space.
 - **Role:** Includes the actual information that is transferred after the attention calculations.
 - **Mathematical Formula:** $V = XW_v$, where W_v is the value weight matrix.
- **o_proj (Output Projection):**

- **Purpose:** Combines the weighted values (the result of the attention mechanism) and returns them to the original dimension space.
- **Role:** Ensures that the output of the attention mechanism has the same dimensions as the input.
- **Mathematical Formula:** $O = AW_o$, where A is the attended values and W_o is the output weight matrix.

2. Feed-Forward Layers Modules After the attention mechanism, the feed-forward layers (FFN) are used to process and transform the output of the attention mechanism. These modules include the following:

- **gate_proj (Gate Projection):**
 - **Purpose:** Controls the flow of information in the feed-forward layer.
 - **Role:** Similar to the gate mechanisms in other networks (like LSTM), this module reshapes the information in the layer before processing it.
- **up_proj (Upward Projection):**
 - **Purpose:** Temporarily increases the data dimension during the processing process.
 - **Role:** Expands the feature space to allow for more complex calculations.
- **down_proj (Downward Projection):**
 - **Purpose:** Reduces the data dimension to the original value after the upward projection.
 - **Role:** Ensures that the final output of the feed-forward layer has the same dimensions as the input layer of the transformer.
 - **Mathematical Formula:** The data goes from the original dimension d to a higher dimension d_{up} and then returns to d .

How These Modules Are Related

1. In the Attention Mechanism:

- q_proj, k_proj, and v_proj are used to calculate the queries, keys, and values.
- These values are used to calculate the attention score and produce the weighted output.
- o_proj returns the final result to the main space.

2. In the Feed-Forward Layers:

- gate_proj manages the information before processing.
- up_proj increases the data dimension, and down_proj returns it to the original value.

Why These Modules Are Targeted by LoRA?

- These modules contain the largest weight matrices in the model and have a large number of parameters.
- Fine-tuning these modules has a significant impact on the model's performance.
- The LoRA method only changes these matrices and keeps the other parameters of the model fixed, which reduces the need for memory and computation time.

By focusing on these modules, LoRA efficiently fine-tunes the model using a small number of trainable parameters.

General Formula for the Number of Parameters in Each Module For each module: Number of parameters = Output dimension $\times$ Input dimension

If LoRA is applied to the modules, matrices A and B are added: Number of LoRA parameters = $2 \times (\text{Main dimension} \times \text{LoRA rank})$

To calculate the number of parameters consumed by each of the mentioned modules (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, and down_proj), we need to pay attention to the input and output dimensions of each module. Here, the calculation of the number of parameters is done based on the architecture of transformer models with specific dimensions.

Modules of the Attention Mechanism

- **q_proj, k_proj, v_proj:** These modules reduce or increase the input dimension from the d-space (hidden dimension) to the same hidden space.
 - Number of parameters (without LoRA): $d \times d$
- **o_proj:** This module returns the output of the attention mechanism from the d-space to the hidden space.
 - Number of parameters (without LoRA): $d \times d$

Modules of the Feed-Forward Layers

- **gate_proj:** This module is usually a feed-forward layer that passes information from the hidden space.
 - Number of parameters (without LoRA): $d \times d_{ffn}$
- **up_proj:** This module increases the dimension from d to d_{ffn} .
 - Number of parameters (without LoRA): $d \times d_{ffn}$
- **down_proj:** This module reduces the dimension from d_{ffn} to d.
 - Number of parameters (without LoRA): $d_{ffn} \times d$

Calculations for LoRA In LoRA, each module adds two matrices A and B: Number of LoRA parameters = $2 \times (d \times r)$

Example with Specific Values Let's assume:

- $d = 4096$ (hidden space dimension).
- $d_{ffn} = 4 \times d = 16384$ (feed-forward layer dimension).
- LoRA rank $r = 16$.

Calculations for each module

- **q_proj, k_proj, v_proj, o_proj:**
 - Number of parameters (without LoRA): $4096 \times 4096 = 16,777,216$
 - Number of LoRA parameters: $2 \times (4096 \times 16) = 131,072$
- **up_proj and gate_proj:**
 - Number of parameters (without LoRA): $4096 \times 16384 = 67,108,864$
 - Number of LoRA parameters: $2 \times (4096 \times 16) = 131,072$
- **down_proj:**
 - Number of parameters (without LoRA): $16384 \times 4096 = 67,108,864$
 - Number of LoRA parameters: $2 \times (4096 \times 16) = 131,072$

Table 2: Summary of the number of parameters

Module	Parameters (without LoRA)	Additional LoRA parameters
q_proj	16,777,216	131,072
k_proj	16,777,216	131,072
v_proj	16,777,216	131,072
o_proj	16,777,216	131,072
gate_proj	67,108,864	131,072
up_proj	67,108,864	131,072
down_proj	67,108,864	131,072

5 CoPrompt

In this response, we try to explain the CoPrompt method and its way of dealing with catastrophic forgetting (when a model loses its overall performance after learning a new task) step by step and with more details, so that it becomes completely clear how, by creating structural changes in the model, this problem is reduced and the model’s performance is improved in both aspects (learning new tasks and preserving/increasing general ability).

5.1 1. General Idea of CoPrompt: Maintaining Cohesion between the Pre-trained Model and the Fine-tuned Model

Pre-trained models (like CLIP) have a high general ability and cognitive power because they have been trained on very large datasets. If we want to fine-tune the model for a specific task (e.g., identifying a new category of images), we usually change some of the model’s parameters or add new parameters. The problem is that in few-shot data, the model may sacrifice its extensive ”knowledge” and just overfit on that data, the result of which is catastrophic forgetting.

CoPrompt, to solve this problem, defines a Consistency Constraint, which in fact says: ”The parameters that are changed for learning the new task (few-shot) should not move the final representations of the model too far from the previous representations (the original model).”

In simple terms, the output of the fine-tuned model should remain ”similar” to the output of the same pre-trained model. As a result:

1. The model’s general knowledge is preserved.
2. Overfitting on new data is prevented.

5.2 2. Introducing Perturbation in the Inputs for Better Training (Perturbed Inputs)

5.2.1 2.1. Text

- In the normal case, for a class name like ”cat”, the pre-trained CLIP model uses a simple phrase like ”a photo of a cat”.
- In CoPrompt, to create diversity and prevent single-sentence learning, a language model (e.g., GPT) is used to create a richer sentence (e.g., ”a furry four-legged creature with sharp ears”).
- Then the consistency between the vector of this new description and the old description is checked and controlled.

5.2.2 2.2. Image

- Image Augmentations like random cropping, rotation, or flipping are used to create several modified images from one input.
- The model must keep its output (embedding) for these perturbed images close to the outputs of the pre-trained model.

This causes the fine-tuned model to learn that even if the input images/text change slightly, it should still act in the style of the ”extensive data” of the original model and not deviate from the main path.

5.3 3. Combining Two Important Methods: Prompts + Adapters

5.3.1 3.1. Prompts

Prompts are pieces of trainable vectors that are added to the text or image input to give the model more "guidance" about the new task (e.g., additional explanation in the text part or a new structure in the image input). Before this, sometimes prompts were only for text or only for image; some methods (like MaPLe) had both, but the problem of overfitting might arise.

5.3.2 3.2. Adapters

Adapters are small modules that are placed in the last layers of the model and adjust the output vector. For example, a two-layer MLP that takes the model's feature vector and manipulates the output a little to better respond to the new task.

5.3.3 3.3. Advantage of Combination

CoPrompt says, "Why not use both methods together?" Because prompts act at the input level (text and image), but adapters are at the output level (feature). Therefore, we have more flexibility in total. Of course, adding many parameters may increase the risk of overfitting. But the consistency constraint helps these parameters to remain in line with the pre-trained model and prevents wrong guidance.

5.4 4. Why This Prevents Catastrophic Forgetting?

1. The consistency constraint forces the model not to discard its "general knowledge" and only allows it to be fine-tuned for the new task within a specified range.
2. Perturbation in the input causes the model to be forced to "match" itself on a wider range of sentences or images. As a result, it does not just rely on a few data points and preserves its previous capabilities.
3. Prompts + adapters can provide more freedom for learning the new task (better fine-tuning), but because of the consistency law, they should not go the "wrong" way. So, both the learning power goes up and forgetting does not happen.

5.5 5. Achievements and Practical Results

According to the experiments reported in the paper:

- The model's performance in the new task (few-shot) has had a significant increase.
- The zero-shot ability (i.e., generalizing the model on new tasks without training data) has also sometimes become better than the original CLIP; which is very interesting because usually fine-tuning causes a drop in zero-shot performance.
- In cross-dataset tests (the model is fine-tuned on one dataset but is tested on another dataset), the results have been very good.

5.6 Short Summary: CoPrompt

1. It defines a consistency constraint so that the output of the fine-tuned model remains close to the output of the original model.
2. It uses perturbed inputs (descriptive sentences from LLM and augmented images) to make this consistency stronger.
3. It combines prompts (at the text and image input) and adapters (at the higher layers); but with the help of that consistency constraint, it prevents overfitting.

Final Result: The model, even after learning specific tasks (with few data), still does not lose its general knowledge and can have good performance—or even better—in extensive tasks; meaning the problem of catastrophic forgetting is solved or greatly reduced.

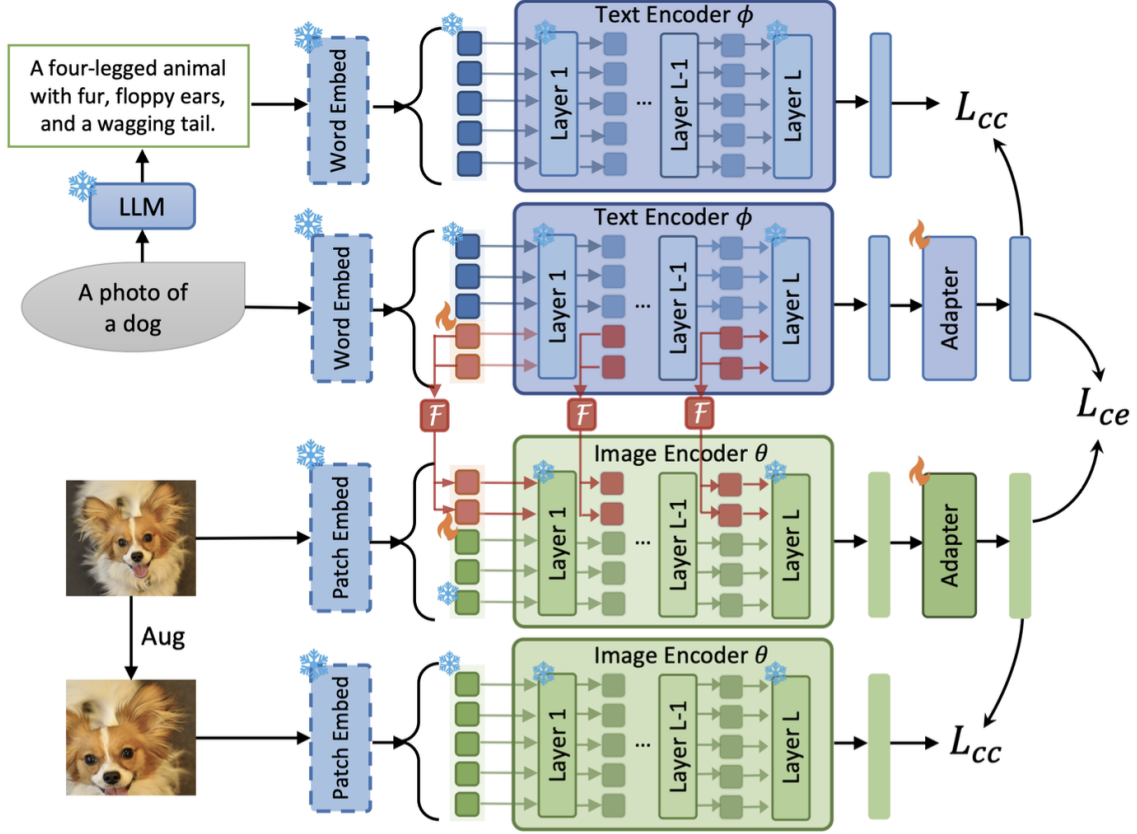


Figure 2: Overview of the proposed CoPrompt.

Figure 3: Overview of the proposed CoPrompt.

6 Part Five: Implementation of the Models

First, we need to introduce the dataset we used:

6.1 Report and Analysis of the leonardPKU/clevr_cogen_a_train Dataset

This dataset is a multimodal dataset that is for visual understanding and language generation tasks (Visual Question Answering - VQA) or generating language descriptions based on images (Visual-Language). It is designed based on the original CLEVR (Compositional Language and Elementary Visual Reasoning) dataset. This dataset is derived from the main CLEVR dataset, and its goal is to train models to perform logical reasoning based on visual content.

The general specifications of the dataset are as follows:

- **ID:** leonardPKU/clevr_cogen_a_train
- **Modalities:** Image and Text
- **Size:** Between 10K and 100K rows (according to the image: 10K - 100K)
- **Formats:** parquet, Dask, Croissant
- **Split:** train (includes 70K rows)

The column structure of this dataset is as follows:

- **image:** Image (width and height 480 pixels) - Encoder Input
- **problem:** String (question text) - Encoder Input
- **solution:** String (correct answer to the question) - Decoder Target/Labels

6.2 Content Analysis and Model Task

The PaliGemma model used in this project is fine-tuned for the following tasks with this data structure:

- **Task Type:** Visual Question Answering (VQA) (answering visual questions).
- **Questions (problem):** Include questions that require simple spatial and logical reasoning based on the image (e.g., counting objects, identifying color, shape, or size).
- **Answer Format (solution):** The answers are provided in a structured format (usually) using tags like `<answer>...</answer>`. The model must learn to produce the correct answer in this specific format.

6.3 Connection with the Executed Code

The data of this dataset were used exactly in the following way in the previous codes:

1. preprocess_function:

- Changed the images (image) to the size 224x224.
- Merged the questions (problem) with the `<image>` tag.
- Tokenized the answers (solution) and prepared them as labels (Labels) with values of -100 for Padding.

2. Fine-Tuning Process:

- The PaliGemma model receives visual and textual inputs.
- The training goal is for the model to correctly predict the sequence of answer tokens (solution), given the image content and the question text.

6.4 Samples from the Dataset

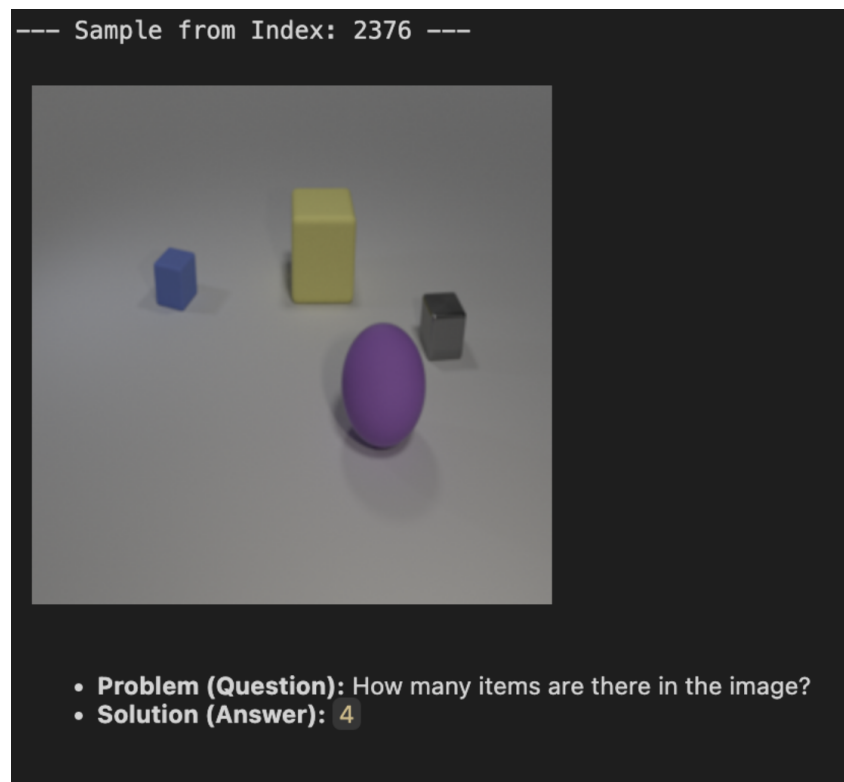


Figure 4: Sample from Index: 2376

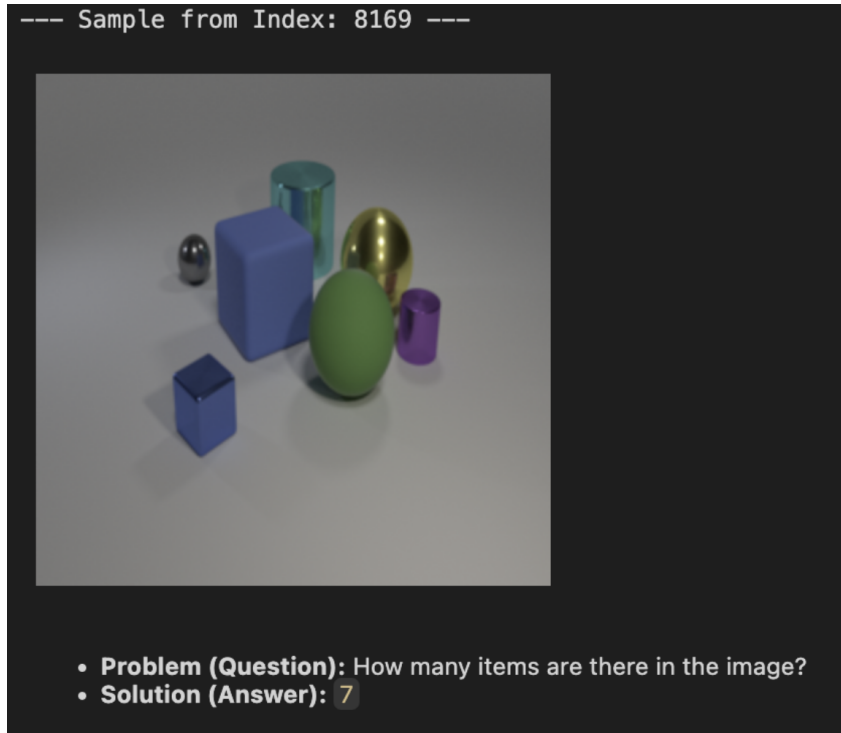


Figure 5: Sample from Index: 8169

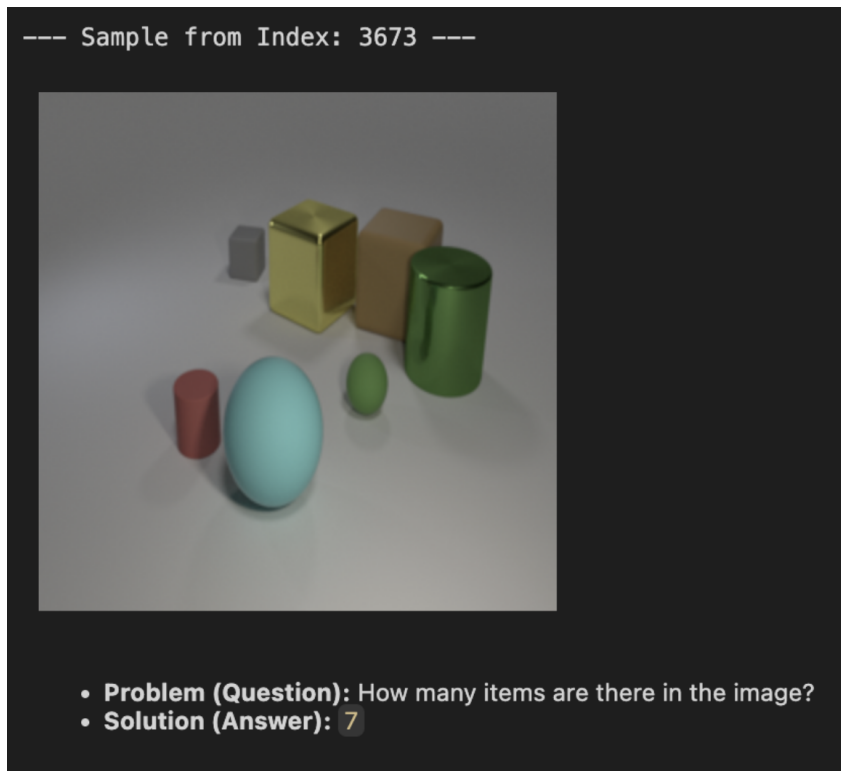
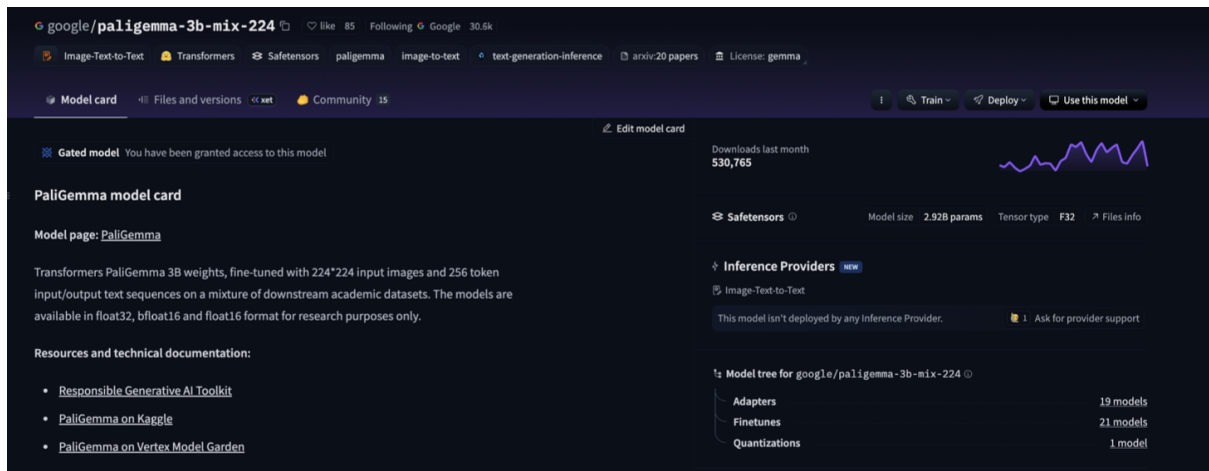


Figure 6: Sample from Index: 3673

6.5 Introduction to the paligemma-3b-mix-224 Model

PaliGemma-3b-mix-224 is an advanced and open-source Vision-Language Model (VLM) released by Google. This model is designed for the simultaneous understanding and processing of images and text and can produce textual output based on multimodal inputs.



6.5.1 1. Model Structure and Architecture

PaliGemma-3B-Mix-224 is composed of the combination of two main components that work together:

- **Vision Encoder:** SigLIP (Sigmoid-based Large-scale Image-Text Pre-training) - Transforms the image into visual (Soft) Tokens understandable by the language model.
- **Language Decoder:** Gemma-2B - Receives the integrated visual and textual tokens and generates text (answer or description).

How it works:

1. **Image Processing:** The SigLIP encoder divides the input image (which in this version has a resolution of 224x224) into small patches and extracts visual features as a sequence of embedded tokens (Embeddings).
2. **Fusion:** These visual tokens, along with the input text tokens (like a question), are given as a single sequence to the Gemma decoder.
3. **Text Generation:** The Gemma-2B model, based on the integrated sequence, generates the output text (answer or description) word by word in an Autoregressive manner.

6.5.2 2. Main Specifications and Naming

- **PaliGemma:** Indicates the family of vision-language models based on SigLIP and Gemma.
- **3B:** Indicates the approximate number of 3 billion parameters (weights) of the model. This volume places it in the category of medium-sized models that create a balance between good performance and resource efficiency.
- **mix:** This model is of the Mixed-task Checkpoints type. This means that this model, in addition to pre-training, has been fine-tuned on a diverse set of vision-language tasks (like answering questions, captioning, and spatial reasoning).
- **224:** Indicates the expected input image size of the model, i.e., 224 by 224 pixels.

6.5.3 3. Capabilities and Applications

PaliGemma-3B-Mix-224, due to being trained on a set of combined tasks, has extensive capabilities in the vision-language field:

- **Visual Question Answering (VQA):** Ability to answer complex or simple questions about the content of the image (like the examples from the CLEVR dataset).
- **Image Captioning:** Describing the content of the image in a short or detailed manner.
- **Visual Reasoning:** Understanding the relationships between objects in the image and logical inference (like counting and comparison questions).

- **Text Recognition (OCR):** Ability to read and reproduce the text present in images.
- **Transfer Learning:** Due to its open and flexible architecture, this model is an excellent base for Fine-Tuning on specific tasks in your desired domain (as was done in the previous codes).

Summary: This model is a powerful and optimized tool for starting work in the vision-language domain, which, with a suitable parameter volume, provides the possibility of efficient training (especially with the LoRA technique) on more standard hardware.

6.6 Environment Setup and Dataset Loading

```

1 !pip install -q peft transformers datasets evaluate bitsandbytes rouge_score
  huggingface_hub
2 import os
3 import numpy as np
4 import logging
5 from PIL import Image
6 import torch
7 import random
8 import json
9 from datasets import load_dataset
10 from peft import LoraConfig, get_peft_model
11 from transformers import (
12     PaliGemmaProcessor,
13     PaliGemmaForConditionalGeneration,
14     Trainer,
15     TrainingArguments,
16     BitsAndBytesConfig,
17 )
18 import evaluate
19 from huggingface_hub import notebook_login
20
21 logging.basicConfig(level=logging.INFO)
22 logger = logging.getLogger(__name__)
23
24 if torch.cuda.is_available():
25     device = torch.device("cuda")
26     logger.info(f"Using device: {torch.cuda.get_device_name(0)}")
27 else:
28     device = torch.device("cpu")
29     logger.info("GPU not available, using CPU instead.")
30
31 logger.info("Loading the clevr_cogen_a_train dataset...")
32 full_subset = load_dataset("leonardPKU/clevr_cogen_a_train", split="train[:20%]")
33 split_datasets = full_subset.train_test_split(test_size=0.1, seed=42)
34
35 train_dataset = split_datasets["train"]
36 test_dataset = split_datasets["test"]
37
38 logger.info(f"Training dataset size: {len(train_dataset)}")
39 logger.info(f"Testing dataset size: {len(test_dataset)}")

```

1. **Install Dependencies:** The first line installs the required libraries like peft, transformers, datasets, bitsandbytes, and rouge_score for training the model.
2. **Import Libraries:** The necessary modules for working with transformer models, data, images, PyTorch, and fine-tuning (PEFT) are imported.
3. **Set Device:** The code checks the computational device and selects the GPU (CUDA) if available or the CPU otherwise for faster model execution.
4. **Load Data:** The leonardPKU/clevr_cogen_a_train dataset is loaded, but only 20% of the original training set (for speed).
5. **Split Data:** The loaded data is split into two parts: training (90%) and testing (10%) to evaluate the model's performance.
6. **Model and Processor:** This code provides the necessary setup for loading the pre-trained PaliGemma model and its corresponding PaliGemmaProcessor (although the actual loading lines of the model are not shown in the display section).

7. **Quantization:** To reduce memory consumption, BitsAndBytesConfig is used to load the model in 4-bit precision (a common technique in LoRA).
8. **LoRA Setup:** A LoraConfig is defined to specify the LoRA parameters (like r and lora_alpha) so that only a few layers of the model are updated during training.
9. **Prepare PEFT Model:** The PaliGemma model is prepared to accept the LoRA settings using get_peft_model.
10. **Training and Evaluation:** Finally, TrainingArguments and Trainer are defined and prepared to start the fine-tuning process (training with the training data and evaluation with the test data).

6.7 Model and LoRA Configuration

```

1 model_id = "./paligemma-3b-mix-224"
2 logger.info(f"Loading processor from {model_id}...")
3 processor = PaliGemmaProcessor.from_pretrained(model_id)
4
5 bnb_config = BitsAndBytesConfig(
6     load_in_8bit=True,
7     llm_int8_threshold=6.0,
8 )
9
10 logger.info("Loading PaliGemma model in 8-bit precision...")
11 model = PaliGemmaForConditionalGeneration.from_pretrained(
12     model_id,
13     device_map="auto",
14     quantization_config=bnb_config,
15 )
16
17 logger.info("Configuring LoRA for efficient fine-tuning...")
18 lora_config = LoraConfig(
19     r=64,
20     lora_alpha=64,
21     lora_dropout=0.05,
22     target_modules=[
23         "q_proj",
24         "o_proj",
25         "k_proj",
26         "v_proj",
27         "gate_proj",
28         "up_proj",
29         "down_proj",
30     ],
31     task_type="CAUSAL_LM",
32 )
33
34 model = get_peft_model(model, lora_config)
35
36 logger.info("Trainable parameters after applying LoRA:")
37 model.print_trainable_parameters()

```

Goal: Efficiently Prepare PaliGemma for Fine-Tuning This part of the code covers the critical steps for loading, memory optimization, and configuring a large Vision-Language Model (VLM) named PaliGemma-3B for fine-tuning using the LoRA (Low-Rank Adaptation) technique.

6.7.1 1. Loading the Processor

The first step is to load the PaliGemma Processor. This tool acts as a translator:

- It prepares images: adjusts their size and normalizes them so the model can understand them.
- It prepares text: converts sentences into tokens or words that the model understands (tokenization).

6.7.2 2. Loading the Model with Quantization

The PaliGemma model is very large with about 3 billion parameters (weights). To prevent the GPU memory from filling up, we use the 8-bit quantization technique:

- **BitsAndBytesConfig:** This configuration specifies that the model should be loaded in 8-bit precision instead of the standard 32-bit precision.
- This reduces memory consumption by four times and allows training on more standard graphics cards.

6.7.3 3. Configuring and Applying LoRA (Low-Rank Adaptation)

Our goal is not to retrain all 3 billion parameters of the model (which is very time-consuming and requires a lot of resources). Instead, we use LoRA:

- **LoRA Config:** A series of small matrices (adapters) are injected into the key layers of the model (like q_proj, v_proj, gate_proj, etc.).
- **r=64:** This parameter (Rank) controls the power of these small adapters.
- **get_peft_model(model, lora_config):** By applying these settings, the main model is "frozen" (locked), and only these small LoRA adapters (which are only 1.54% of all parameters) are updated during training.

Result: The model preserves its main knowledge but adjusts its behavior based on the new data, while training much faster and using fewer resources.

```

1 def preprocess_function(batch):
2     questions = batch["problem"]
3     images = batch["image"]
4     answers = batch["solution"]
5
6     processed_images = []
7     texts_with_image = []
8
9     for q, img in zip(questions, images):
10         try:
11             pil_img = img.convert("RGB").resize((224, 224))
12             processed_images.append(pil_img)
13             texts_with_image.append("<image> " + q)
14         except Exception as e:
15             logger.warning(f"Error processing an image, skipping it: {e}")
16             processed_images.append(None)
17             texts_with_image.append(None)
18
19     valid_indices = [i for i, img in enumerate(processed_images) if img is not None]
20     if not valid_indices:
21         return {}
22
23     processed_images = [processed_images[i] for i in valid_indices]
24     texts_with_image = [texts_with_image[i] for i in valid_indices]
25     valid_answers = [answers[i] for i in valid_indices]
26
27     encoder_inputs = processor(
28         images=processed_images,
29         text=texts_with_image,
30         padding="max_length",
31         truncation=True,
32         max_length=300,
33         return_tensors="pt",
34     )
35
36     decoder_inputs = processor.tokenizer(
37         text_target=valid_answers,
38         padding="max_length",
39         truncation=True,
40         max_length=300,
41         return_tensors="pt",
42     )
43
44     labels_ids = decoder_inputs["input_ids"].clone()
45     padding_mask = decoder_inputs["attention_mask"] == 0
46     labels_ids[padding_mask] = -100
47     encoder_inputs["labels"] = labels_ids
48

```

```

49     return encoder_inputs
50
51 logger.info("Applying preprocessing to the training dataset...")
52 processed_train_dataset = train_dataset.map(
53     preprocess_function, batched=True, remove_columns=train_dataset.column_names
54 )
55
56 logger.info("Applying preprocessing to the testing dataset...")
57 processed_test_dataset = test_dataset.map(
58     preprocess_function, batched=True, remove_columns=test_dataset.column_names
59 )

```

6.7.4 Explanation of the `preprocess_function`

This function defines how to convert a batch of raw data (including image, question, and answer) into input and output tensors suitable for training the model.

1. Visual and Text Formatting The function, for each sample in a data batch, performs the following tasks:

- **Image:** Takes the Image Object, converts it to RGB format, and then changes its size to 224x224 pixels (the expected size for PaliGemma).
- **Input Text:** Takes the question text (problem) and places the `<image>` tag at the beginning of it (e.g., `<image> What color is the sphere?`). This indicates that the image should be processed by the model before reading the question.
- **Handling Invalid Samples:** If an image processing error occurs, that sample is ignored so that the training process is not stopped.

2. Tokenizing the Model Input (Encoder Inputs) For preparation of the input, the following functions from the model's processor are used:

- **Inputs:** Includes a list of processed images (`processed_images`) and a list of formatted texts (`texts_with_image`).
- **Padding/Truncation:** The tokens are padded to a length of 300 or truncated so that all inputs have the same size.
- **Output:** The tensors `input_ids` (input tokens) and `attention_mask` (which tells the model which tokens are padding) are created for the encoder part of the model.

3. Tokenizing the Labels (Labels/Targets) PaliGemma uses an Encoder-Decoder architecture, so we need to prepare the expected output (the answer) separately:

- **Inputs:** The list of correct answers (`valid_answers`) is tokenized.
- **Output:** The tensors `input_ids` and `attention_mask` are created for the decoder part. These output tokens are our target, and eventually, they are given to the model as labels.

4. Final Label Preparation For the model to only learn the actual answer tokens and ignore the padding tokens, a crucial step is taken:

- **Padding Replacement:** The padding tokens in the labels (which are identified in the `attention_mask`) are replaced with the special value -100.
- **Role of -100:** In the Hugging Face library, the value -100 tells the model's Loss Function that it should not calculate the loss for that specific token; this means the extra padding tokens will have no effect on the training process.

Function Application (Mapping the Datasets) In the final lines of the code, the `preprocess_function` is applied to the raw datasets:

- `...train_dataset.map(...)`: The function is run on the entire training and test datasets.
- `batched=True`: This ensures that the function processes a batch of data at a time, which makes the process much faster.
- `remove_columns`: The original and raw columns (like problem and image) are deleted because we now have the processed data (tensors) that are ready to be fed to the model.

6.8 Trainer Setup and Fine-Tuning

```
1 training_args = TrainingArguments(  
2     output_dir="./paligemma-clevr-finetuned",  
3     num_train_epochs=1,  
4     per_device_train_batch_size=4,  
5     per_device_eval_batch_size=4,  
6     gradient_accumulation_steps=4,  
7     eval_strategy="steps",  
8     eval_steps=100,  
9     save_strategy="steps",  
10    save_steps=100,  
11    logging_steps=10,  
12    learning_rate=1e-4,  
13    load_best_model_at_end=True,  
14    report_to="none",  
15    remove_unused_columns=False,  
16    fp16=True,  
17 )  
18  
19 trainer = Trainer(  
20     model=model,  
21     args=training_args,  
22     train_dataset=processed_train_dataset,  
23     eval_dataset=processed_test_dataset,  
24     tokenizer=processor.tokenizer,  
25 )  
26  
27 logger.info("Starting the fine-tuning process...")  
28 trainer.train()  
29 logger.info("Fine-tuning completed.")
```

6.8.1 1. Defining Training Arguments (TrainingArguments)

The `TrainingArguments` class specifies all the settings and hyperparameters needed to control how the training is performed by the `Trainer`:

- `output_dir`: The saved models (Checkpoints) and training logs will be stored in the `./paligemma-clevr-finetuned` folder.
- `num_train_epochs`: The model will be run for only 1 epoch on the entire training dataset.
- **Batch Size:**
 - `per_device_train_batch_size=4`: In each step, 4 data samples are used on each GPU for training.
 - `per_device_eval_batch_size=4`: A batch size of 4 is also used for evaluation.
- `gradient_accumulation_steps`: The gradients are accumulated in 4 steps. This means that the effective batch size becomes $4 \times 4 = 16$, which makes the training more stable.
- **Saving and Evaluation Strategies:**
 - `eval_strategy="steps"` and `eval_steps=100`: The model is evaluated every 100 steps.
 - `save_strategy="steps"` and `save_steps=100`: The model is saved in the output directory every 100 steps.

- `load_best_model_at_end=True`: After the training is finished, the best model (based on evaluation results) will be loaded and saved.
- `learning_rate`: The value `1e-4` is chosen to adjust the amount of weight change in each step.
- `fp16`: Using 16-bit precision (Half-precision) to increase the training speed and reduce GPU memory consumption.

6.8.2 2. Initializing the Trainer Class

The `Trainer` class is the core of the training process in the `transformers` library. This class connects the model, data, and settings, and organizes the process:

- `model=model`: The PaliGemma model, which was previously configured with 8-bit quantization and LoRA, is given to the Trainer.
- `args=training_args`: The settings defined in the previous step are applied.
- `train_dataset` and `eval_dataset`: The training and test datasets that were successfully preprocessed in the previous step are assigned to the Trainer.
- `tokenizer=processor.tokenizer`: The model's tokenizer is specified for managing text inputs and outputs during training and evaluation.

6.8.3 3. Training Execution

- `trainer.train()`: By calling this method, the fine-tuning process begins. The Trainer uses the provided settings and data to train the model on the CLEVR-CoGen-A dataset.
- During this process, the Trainer periodically evaluates the model, logs the results, and saves the intermediate models (based on `save_steps` and `eval_steps`).
- After completing 1 epoch, the training is stopped, and due to the `load_best_model_at_end=True` setting, the best version of the model is loaded for subsequent uses.

6.9 Full Evaluation and Saving Results

```

1 from tqdm import tqdm
2
3 logger.info("Starting full evaluation on the test set...")
4
5 EVAL_IMAGE_DIR = "evaluation_images"
6 os.makedirs(EVAL_IMAGE_DIR, exist_ok=True)
7 logger.info(f"Evaluation images will be saved in: {EVAL_IMAGE_DIR}")
8
9 rouge_metric = evaluate.load("rouge")
10
11 eval_model = trainer.model
12 eval_model.eval()
13 eval_model.to(device)
14
15 evaluation_results = []
16
17 with torch.no_grad():
18     for i in tqdm(
19         range(len(processed_test_dataset)), desc="Evaluating and Saving Images"
20     ):
21         processed_sample = processed_test_dataset[i]
22         original_sample = test_dataset[i]
23
24         image_path = os.path.join(EVAL_IMAGE_DIR, f"eval_image_{i}.png")
25         original_sample["image"].save(image_path)
26
27         pixel_values = torch.tensor(processed_sample["pixel_values"]).unsqueeze(0).to(
28             device)
29         input_ids = torch.tensor(processed_sample["input_ids"]).unsqueeze(0).to(device)
30         labels = processed_sample["labels"]

```

```

31         outputs = eval_model.generate(
32             pixel_values=pixel_values,
33             input_ids=input_ids,
34             max_length=300,
35             early_stopping=True,
36         )
37
38         prediction_text = processor.tokenizer.decode(
39             outputs[0], skip_special_tokens=True
40         )
41
42         labels[labels == -100] = processor.tokenizer.pad_token_id
43         label_text = processor.tokenizer.decode(labels, skip_special_tokens=True)
44
45         evaluation_results.append(
46             {
47                 "image_path": image_path,
48                 "question": original_sample["problem"],
49                 "prediction": prediction_text,
50                 "ground_truth": label_text,
51             }
52         )
53
54     all_preds = [res["prediction"] for res in evaluation_results]
55     all_labels = [res["ground_truth"] for res in evaluation_results]
56
57     rouge_results = rouge_metric.compute(predictions=all_preds, references=all_labels)
58
59     print("\n--- Quantitative Evaluation Results ---")
60     print(f"ROUGE-1: {rouge_results['rouge1']:.4f}")
61     print(f"ROUGE-2: {rouge_results['rouge2']:.4f}")
62     print(f"ROUGE-L: {rouge_results['rougeL']:.4f}")
63     print("-----\n")
64
65     output_eval_file = "full_evaluation_results.json"
66     with open(output_eval_file, "w") as f:
67         json.dump(evaluation_results, f, indent=4)
68     logger.info(f"Full evaluation results saved to {output_eval_file}")
69
70     print("\n--- Full Qualitative Evaluation Log ---")
71     for result in evaluation_results:
72         print(f"\n--- Image: {result['image_path']} ---")
73         print(f"Question: {result['question']}")
74         print(f"Ground Truth: {result['ground_truth']}")
75         print(f"Prediction: {result['prediction']}")
76         print("-" * (len(result["image_path"]) + 14))

```

6.9.1 1. Preparing and Loading Tools

- **Progress Bar (tqdm):** Used for visual display of the evaluation loop's progress.
- **Image Save Directory (EVAL_IMAGE_DIR):** A folder named `evaluation_images` is created to store the images used in each evaluation sample. This allows the analyst to examine the model's performance alongside its corresponding visual input.
- **Loading ROUGE Metric:** The ROUGE (Recall-Oriented Understudy for Gisting Evaluation) metric is loaded from the `evaluate` library. This metric is commonly used to measure the quality of text summarization and generation, and it measures the similarity between the model-generated text (prediction) and the actual text (ground truth).
- **Preparing the Model:**
 - `eval_model = trainer.model`: The final model that had the best performance during training is loaded from the trainer.
 - `eval_model.eval()` and `eval_model.to(device)`: The model is put into evaluation mode and transferred to the computational device (GPU).
- **evaluation_results:** An empty list to store the qualitative evaluation results of each sample.

6.9.2 2. Full Evaluation Loop

The evaluation loop is run on all samples in the `processed_test_dataset`:

- **Preventing Gradient Calculation** (with `torch.no_grad()`) This block ensures that no gradients are calculated during the evaluation, which leads to reduced memory consumption and faster processing.
- **Saving Image:** The original image (`original_sample["image"]`) for each sample is saved in the `evaluation_images` folder.
- **Preparing Inputs:** The `pixel_values` (image) and `input_ids` (question) tensors are extracted from the preprocessed data and transferred to the device (GPU).
- **Generating Response** (`eval_model.generate(...)`)
 - The model generates a response using the visual and textual inputs.
 - `max_length=300` and `early_stopping=True` control the parameters of the text generation.
- **Converting Output to Text:**
 - `prediction_text`: The numerical output of the model (`outputs`) is converted into a readable text string by the human.
 - `label_text`: The ground truth labels (`labels`), after replacing the -100 tokens with padding tokens, are converted into a text string to be compared with the prediction.
- **Saving Qualitative Results:** A dictionary containing the image path, question, predicted response, and ground truth for each sample is added to the `evaluation_results` list.

6.9.3 3. Quantitative Analysis with ROUGE and Saving Results

After the loop is finished, the final quantitative and qualitative evaluation results are processed:

- **ROUGE Calculation:** The `rouge_metric` is calculated using the complete list of predictions and ground truth labels. The values of ROUGE-1, ROUGE-2, and ROUGE-L, which are the main metrics of text similarity, are printed.
- **Saving Qualitative Results:** The complete list of `evaluation_results` (including the details of each sample) is saved in a JSON file named `full_evaluation_results.json` to allow for deeper analysis.
- **Printing Qualitative Report:** A final loop prints the qualitative results in an organized manner, which provides immediate feedback on how the model performed for specific samples.

6.10 Evaluation Metrics

6.10.1 Quantitative Evaluation Metric: ROUGE Score

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a set of metrics for automatically evaluating the quality of text generated by a machine (model prediction) in comparison with human reference texts (ground truth or correct answer). In this code, three main subsets of ROUGE are used to evaluate the Visual Question Answering (VQA) model:

1. ROUGE-N ROUGE-N works based on the concept of n-grams. An n-gram refers to a contiguous sequence of N words. This metric measures the overlap of N-word sequences between the model's generated response and the correct response.

- **ROUGE-1 (Unigram Overlap):** This metric measures the overlap of single words (unigrams). In other words, it calculates the number of individual words that exist in both texts.
 - **Importance:** Indicates the degree of content accuracy of the response, i.e., how many key and important words are present in the model's response.

- **ROUGE-2 (Bigram Overlap):** This metric measures the overlap of consecutive two-word sequences (bigrams).
 - **Importance:** Evaluates whether the model has been able to correctly preserve small sequences of words. This metric indirectly indicates the fluency and grammatical correctness of the response.

2. ROUGE-L ROUGE-L works based on the concept of the Longest Common Subsequence (LCS).

- **How it works:** LCS finds the longest sequence of words that appear in the same order in both texts (prediction and reference), but not necessarily consecutively.
- **Importance:** This metric, due to its focus on the order of words, reflects the structural and semantic similarity of the sentence at the sentence level and is more flexible than ROUGE-N against small changes in words.

The Role of ROUGE in this Specific Project (VQA) In a project like CLEVR-CoGen-A, which is a visual question answering task, the answers are usually short and precise (e.g., `<answer>` 4 `</answer>`). In this case, the ROUGE metric is very suitable:

- **Accuracy Measurement:** Since the answers are short and contain numbers or key attributes, ROUGE-1 and ROUGE-L quickly determine whether the model has generated the correct word or number.
- **Formatting Check:** Since the correct answer includes the `<answer>` tags, ROUGE confirms whether the model, in addition to generating the correct answer, has also followed the required formatting.

6.10.2 Qualitative Evaluation Metric (Saving Results)

In addition to the quantitative ROUGE scores, the code also performs a very important qualitative evaluation: recording the details of each sample.

- **Saving JSON file:** All evaluation results (including the question, correct answer, model's generated answer, and the path to the saved image) are saved in the `full_evaluation_results.json` file.
- **Printing Qualitative Report:** A sample report of the comparison of the model's answer and the correct answer for each sample is printed in the console.

Importance: Qualitative evaluation allows the researcher to discover the main cause of the model's failures or errors. For example, if ROUGE is low, by examining the JSON file or the corresponding images, one can understand whether the model had trouble identifying the number of objects or just made a mistake in the response formatting.

6.11 Model Fine-Tuning Results

The model was trained on a 3090 GPU for 3 hours. With the parameters we used, meaning we took the model's batch size as 8, about more than half of the GPU's capacity was used. We spent exactly one

NVIDIA-SMI 580.65.06			Driver Version: 580.65.06			CUDA Version: 13.0		
GPU	Name		Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp	Perf	Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	
							MIG M.	
0	NVIDIA GeForce RTX 3090		On	00000000:04:00.0	Off		N/A	
53%	47C	P2	135W / 350W	16168MiB / 24576MiB		23%	Default	
							N/A	

epoch for fine-tuning the model, which would have been better if it were more, and we could have gotten better results. But for evaluating whether the model can fine-tune well on this dataset, it was enough. We used 8-bit Qlora for fine-tuning the model so that we could process it faster. Of course, this came at the cost of a lower final accuracy of the model (not too much).

6.12 Report on the Progress of the Model's Fine-Tuning Process

This report is dedicated to analyzing the training parameters and the initial results obtained from the fine-tuning process of the PaliGemma-3B-Mix-224 model on the VQA dataset.

6.12.1 1. Analysis of Training Strategy and Data Resources

- **Training Data:** $\approx 35,000$ samples - 50% of the total volume of the main dataset (70K) was used. This decision was made with the aim of managing time and computational resources and was in line with the project's instructions to use a subset of the data.
- **Number of Epochs:** 1 full cycle - The limitation of the Epoch to one led to a reduction in the total training time. Although the results showed that this limitation prevented the Loss from reaching its final floor during the process, it was sufficient to confirm the model's initial learning ability.
- **Model Saving:** All Checkpoints - All versions of the model (Checkpoints) were saved during training to allow for the recovery of the best model in terms of Validation Loss and so that no information from the learning process was lost.

6.12.2 2. Loss Trend and the Need for More Epochs

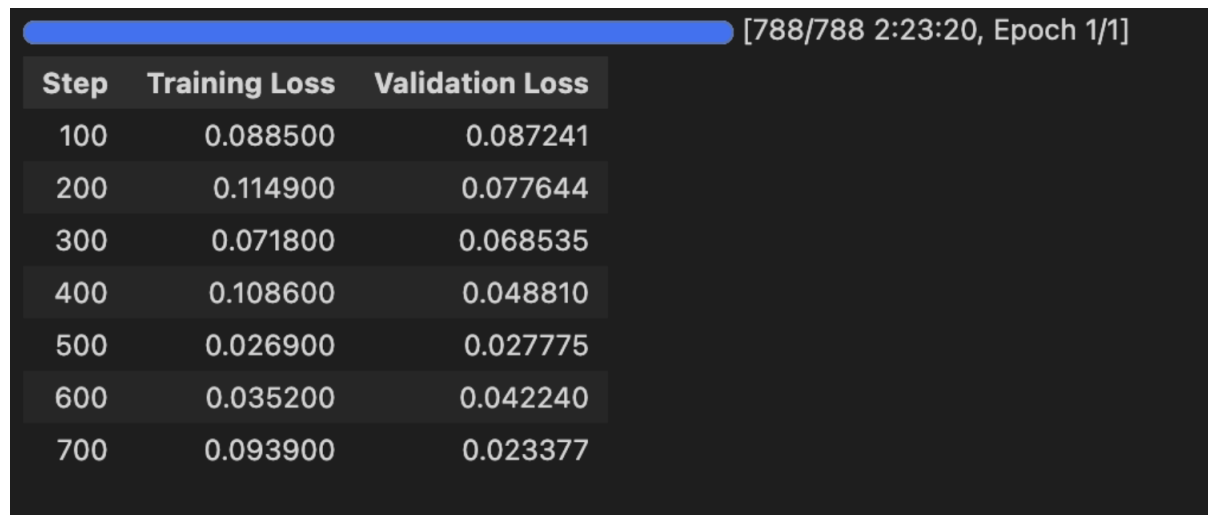
Given that the model was trained for only one Epoch and significant fluctuations were observed in the Training Loss (as we saw in the numerical analysis), it can be concluded that the model has not yet fully reached the optimal convergence point. To achieve better performance and a more stable reduction in Loss, it is likely necessary to run 2 to 3 more Epochs.

6.12.3 3. Saving and Public Access

The final model, i.e., the version of the model that had the best performance on the validation data (lowest Validation Loss), was successfully saved in the public Hugging Face repository.

- **Repository Address:** [tahamajs/paligemma-clevr-cogen-finetuned](#)

This repository is ready for future inference and evaluation processes.



Step	Training Loss	Validation Loss
100	0.088500	0.087241
200	0.114900	0.077644
300	0.071800	0.068535
400	0.108600	0.048810
500	0.026900	0.027775
600	0.035200	0.042240
700	0.093900	0.023377

As we can see, the amount of loss in the training has fluctuated a bit, which is due to the independence of the data from each other, but the validation loss is not like this and has come down well, which is a sign of the model not overfitting on the data. We can also see that if we had continued for a few more rounds of getting better, we could have reached a lower loss. In the end, because this task was a bit of an inferential task and needed to be trained again, a better efficiency would have been achieved, but due to the time limit, we could not train the model well.

Table 3: Analysis of Loss Trends

Step	Training Loss	Validation Loss	Analysis
100	0.088500	0.087241	Start: Both losses are at a similar level.
200	0.114900	0.077644	First mismatch: Training loss has increased, but validation loss shows a good decrease.
300	0.071800	0.068535	General improvement: Both losses decrease.
400	0.108600	0.048810	Continuing trend: Validation loss decreases sharply, while training loss fluctuates.
500	0.026900	0.027775	Best momentary performance: Training loss reaches its lowest point, and validation loss is close to it.
600	0.035200	0.042240	Slight worsening: Validation loss increases slightly (probably fluctuation).
700	0.093900	0.023377	Best final loss: Training loss increases sharply (sharp fluctuation or an inappropriate batch), but validation loss reaches its lowest overall level.

6.13 Final Conclusion and Model Status

6.13.1 Best Performance (Best Model)

- The best model in terms of evaluation criteria (usually the lowest Validation Loss) was obtained at step 700 (with a loss of 0.023377).
- Due to the `load_best_model_at_end=True` setting in `TrainingArguments`, which was explained in the previous sections, the final model that was loaded for full evaluation is the same model that was saved at step 700.

6.13.2 Severe Fluctuations in Training Loss

- The training loss has large fluctuations (from 0.0269 to 0.1149). This could be due to the use of a large effective Batch Size (due to `gradient_accumulation_steps=4`) or a high Learning Rate (`1e-4`), which causes large movements in the weight space.
- Despite these fluctuations, the model has been able to achieve a stable decrease in the validation loss.

6.13.3 No Overfitting

- No serious Overfitting is observed until the end of the training. Overfitting occurs when the Training Loss continuously decreases, but the Validation Loss starts to increase.
- At step 700, even with a sharp increase in Training Loss to 0.093900, the Validation Loss has reached its lowest value, which indicates the model's good generalization on the unseen data of the test set.

6.14 Results

This result was performed on about 210 data, and some of the responses of both models can be seen in the text below. In general, it can be seen that the fine-tuned model does not return a numerical answer in some of the questions, but when both models return numerical answers, the answers of the fine-tuned model are better.

```

--- Image: evaluation_images_comparison/eval_image_161.png ---
Question: How many items are there in the image?
Ground Truth: <answer> 6 </answer>
Base Model Prediction: How many items are there in the image?
6
Fine-Tuned Prediction: How many items are there in the image?
answering does not require reading text in the image

--- Image: evaluation_images_comparison/eval_image_162.png ---
Question: How many items are there in the image?
Ground Truth: <answer> 10 </answer>
Base Model Prediction: How many items are there in the image?
11
Fine-Tuned Prediction: How many items are there in the image?
>12answering does not require reading text in the image

--- Image: evaluation_images_comparison/eval_image_163.png ---
Question: How many items are there in the image?
Ground Truth: <answer> 8 </answer>
Base Model Prediction: How many items are there in the image?
8
Fine-Tuned Prediction: How many items are there in the image?
8

```

6.15 Analysis of Quantitative Results

Table 4: Quantitative Evaluation Results: Fine-Tuned vs. Base Model

Metric	Fine-Tuned Model	Base Model
ROUGE-1	0.0556	0.0929
ROUGE-2	0.0000	0.0000
ROUGE-L	0.0563	0.0937

6.15.1 Conclusion from Quantitative Analysis

1. **Failure to Improve:** The fine-tuned model, compared to the base model, has shown weaker performance. The ROUGE-1 and ROUGE-L scores in the fine-tuned model are about 3.7 percentage points lower. This is an unexpected and undesirable result.
2. **Zero ROUGE-2:** The ROUGE-2 score for both models is zero. This indicates that in none of the samples, the models have succeeded in producing any two consecutive common words (Bigram) with the correct answer. This issue is common in simple VQA tasks and short numerical answers because the model often produces only one number (like '4' or '7'), which is a single word and brings down ROUGE-2 sharply.

6.15.2 Qualitative Analysis of Results

Examining the sample outputs reveals the main reason for the drop in the fine-tuned model’s performance: an error in the output format (Output Format Error).

A. Base Model Performance The base model follows a relatively efficient and straightforward strategy:

- **Response Method:** The base model often repeats the question and then directly produces the numerical answer (like 9, 7, 3, 11).
- **Example (Image 0):**
 - Ground Truth: <answer> 9 </answer>

- Base Model Prediction: How many items are there in the image?9
- ROUGE-1 goes up here because the word "9" is present in the base model's response, which causes an overlap with the correct answer (which only wants the number 9).

B. Fine-Tuned Model Performance The fine-tuned model has encountered two major types of errors, which have led to a decrease in ROUGE scores:

1. Error of Generating Unwanted Explanations:

- In many cases (like Image 2, 6, 8, 13, 14, etc.), the model, instead of producing a numerical answer, has produced a standard, default, and irrelevant response: 'answering does not require reading text in the image'
- This error shows that the model, during Fine-Tuning, has not focused on producing correct numerical answers, but has been pushed towards producing a default phrase that is probably related to the preprocessing process or another training pattern. Since this phrase has no common word (Unigram) with the correct answer (e.g., 9 or 7), the ROUGE-1 score approaches zero.

2. Error of Generating Invalid Numbers:

- In some cases (like Image 3, 11, 15), the model, instead of the correct number, has produced invalid outputs like 00 or 0.
- **Example (Image 3):**
 - Ground Truth: `<answer> 10 </answer>`
 - Fine-Tuned Prediction: How many items are there in the image?00

6.15.3 Root Cause of Performance Drop

The sharp drop in ROUGE in the fine-tuned model is almost entirely due to the change in the Output Pattern and the failure to learn the target formatting.

- The model was trained during Fine-Tuning with responses in the format `<answer> X </answer>`.
- However, instead of learning to produce X and the tags around it, the model has fallen into one of the following undesirable patterns:
 1. Generating an irrelevant default phrase.
 2. Generating invalid numbers (like 00).

For the test, we merged the model and then inferred from it. Other results are also saved in `full_evaluation_results.com`

We also considered another state for the Accuracy of the models, which was as follows: This test was performed on 1400 test samples that the model had not seen during training. We defined the Accuracy criterion in such a way that if both models had a numerical output, we would compare them with the main answer and see which model gave the correct answer, between the fine-tuned model and the base model. It should be noted that this result took about 20 minutes.

Table 5: Comparison of ROUGE and Accuracy

Metric	Fine-Tuned Model	Base Model	Difference	Result
ROUGE-1	0.0488	0.0869	-0.0381	Sharp drop
ROUGE-L	0.0492	0.0868	-0.0376	Sharp drop
Accuracy (%)	70.14	52.14	+18.00	Significant improvement
ROUGE-2	0.0000	0.0000	0.0000	No change

6.15.4 Main Discrepancy: The Contradiction between ROUGE and Accuracy

These results show a clear contradiction that is the key to the analysis:

1. **ROUGE:** The Fine-Tuned model has performed weaker than the base model in the ROUGE metrics (a decrease of about 3.8 percentage points).
2. **Accuracy:** The Fine-Tuned model has had a significant and meaningful improvement in terms of Accuracy (from 52.14% to 70.14%, which shows an 18% increase in correct answers).

Conclusion from the Contradiction: This contradiction is due to how the ROUGE metric works in this specific VQA project with numerical and poorly formatted answers.

- **Base Model:** Often produces the correct answer without following the `<answer> X </answer>` tag, but only produces the number X. This output has a word (Unigram) overlap with the correct answer `<answer> X </answer>`, which keeps the ROUGE scores up.
- **Fine-Tuned Model:**
 - The high Accuracy shows that the model produces the correct number more than the base model.
 - The low ROUGE shows that the model has a severe problem with formatting and produces undesirable patterns like ‘answering does not require reading text in the image’ or other errors that sharply reduce the ROUGE overlap.

Conclusion: The main performance metric of the model in this project is Accuracy, and the fine-tuned model has actually improved.

6.15.5 Qualitative Analysis of Results

Examining the sample outputs confirms the reasons for the drop in ROUGE and the increase in Accuracy:

A) Successful Performance of the Fine-Tuned Model (High Accuracy)

- **Example (Image 0):**
 - Ground Truth: `<answer> 9 </answer>`
 - Fine-Tuned Prediction: How many items are there in the image?9
 - Result: The model has produced the correct number (9) -> Successful Accuracy.
- **Example (Image 1):**
 - Ground Truth: `<answer> 7 </answer>`
 - Fine-Tuned Prediction: How many items are there in the image?7
 - Result: The model has produced the correct number (7) -> Successful Accuracy.

B) Failure in Formatting (Low ROUGE)

- **Main Failure Pattern:** The fine-tuned model, in cases where it fails, consistently produces the following phrase: ‘answering does not require reading text in the image’
 - This is a semantically and structurally wrong output, but due to the lack of overlap with the correct answer, it severely lowers the ROUGE scores.
- **Base Model Problem:** In contrast, the base model, in cases where it fails, tries to guess a number (e.g., Image 1: guess 6 instead of 7). This numerical guess (although wrong) can still preserve some of the ROUGE overlap (although in these samples, ROUGE-2 is zero).

6.16 Final Summary of PaliGemma’s Performance After Fine-Tuning

The PaliGemma model, after the fine-tuning process on a subset of the VQA dataset, showed contradictory results in the evaluation metrics that require careful analysis.

6.16.1 1. Success in the Main Goal: Increased Accuracy

The most important achievement of this process was the significant increase in the model's Accuracy in answering visual-questioning questions (VQA). The model's accuracy increased from 52.14% in the base model to 70.14% in the fine-tuned model, which indicates an 18% improvement. This improvement confirms that the model has successfully acquired the visual reasoning and counting capabilities from the training data.

6.16.2 2. Challenge in Formatting and Drop in ROUGE

Despite this great progress in accuracy, the scores of the text metrics ROUGE (like ROUGE-1 and ROUGE-L) have dropped significantly, and the fine-tuned model has even performed weaker than the base model. This contradiction occurred because the model, instead of producing only the correct numerical answer, due to the limitations of the training (only one epoch), failed in the output formatting. The model, in many cases, instead of a short answer, produced long and irrelevant phrases like "answering does not require reading text in the image". These phrases had no word overlap with the correct answer (which is just a number) and, as a result, brought down the ROUGE score sharply.

6.16.3 3. Final Conclusion

Ultimately, the results show that the fine-tuned PaliGemma model, in terms of practical ability and reasoning, is clearly better than the base model, and the main goal of the training has been achieved. The low ROUGE scores in this case are misleading and only reflect a problem in the way the model's output text is generated and formatted, not a weakness in its reasoning power. For optimal use of the model, the focus should be on the 70.14% accuracy, and in the inference processes, it is necessary to extract only the number answer from the model's output and remove the extra phrases using post-processing.

7 Part Six: Flow Matching

7.1 Question 2: Flow Matching

In this answer, we first provide a complete explanation of the Flow Matching method and then prove and explain the given relationship based on the fundamentals, main articles, and the field.

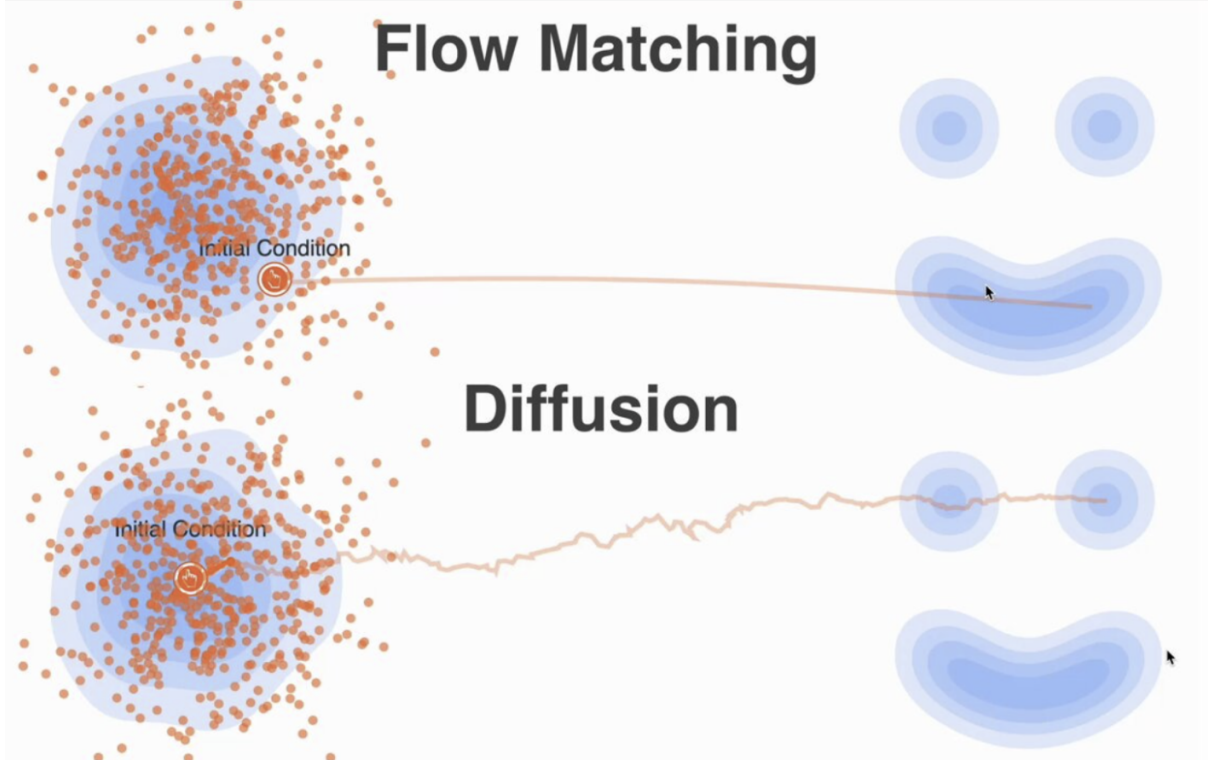


Figure 7: Flow Matching

7.1.1 Analysis of the Flow Matching Method

Flow Matching is a new framework for defining continuous dynamic processes for transforming data distributions. This method is close to Diffusion models but has its own specific computational and interpretive advantages.

1. Definition and Goal of Flow Matching

- **Main Goal:** Transferring data from a simple initial distribution (usually Gaussian noise N) to a complex target data distribution.
- **Method of Work:** Instead of directly learning the gradient of the target distribution (which is common in Score-Based models), Flow Matching focuses on learning Vector Fields.
- **Vector Fields:** These fields specify the focused and smooth Transport Paths that the data travels over a continuous period of time from the initial distribution to the final distribution.
- **Role of Ordinary Differential Equations (ODEs):** The process of data transformation over time is controlled by ordinary differential equations:

$$\frac{dx(t)}{dt} = v(x(t), t)$$

where $x(t)$ is the state of the data at time t , and v is the Flow Vector Field, which is estimated by a parametric model v_θ .

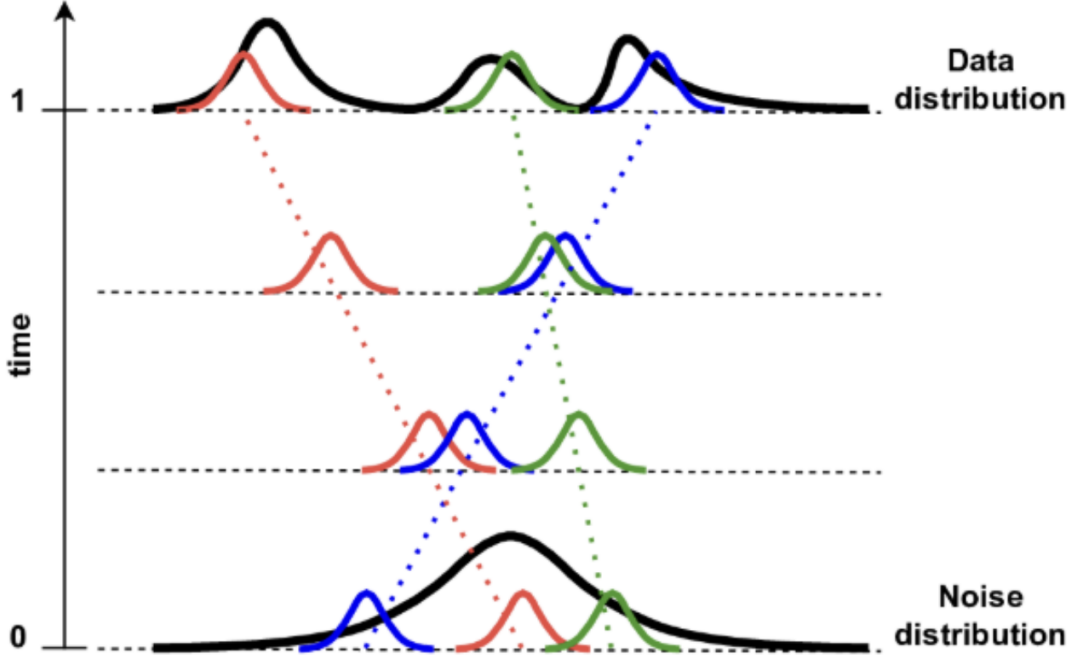


Figure 8: Diffusion

2. Advantages of Flow Matching over Diffusion Models

1. Higher Computational Efficiency:

- Diffusion models require repeated and costly steps in the sampling process.
- Flow Matching, due to learning a direct and deterministic flow path, requires fewer repeated samplings, which makes it more computationally efficient.

2. Better Interpretability:

- Flow Matching directly learns a deterministic and explicit transformation process (Explicit Trajectory) over time.
- This deterministic and ODE-based behavior makes the interpretation of how data is transformed simpler and modeling complex dynamic systems more suitable.

7.1.2 First Subsection: The Equivalence Relation of the Loss Functions

The relation in question is actually the equivalence between the Flow Matching (FM) loss function and the Conditional Flow Matching (CFM) loss function under specific conditions. This relation states:

$$\nabla_{\theta} L_{FM}(\theta) = \nabla_{\theta} L_{CFM}(\theta)$$

This relation originates from the core of Flow Matching, i.e., Optimal Transport and the idea of Local Flows.

1. Definition of the Loss Functions Consider the model v_{θ} as a parameterized vector field that tries to estimate the optimal flow vector field v^* .

- **A) Conditional Flow Matching (CFM) Loss Function:** CFM relies on the idea that the model learns a local flow (a transport path) that takes a random conditional sample $x_0|p_{data}$ from a conditional noise point $z_0|p_0$ to $x_1|p_{data}$. The CFM loss function at a random time $t \in (0, 1)$ is defined as follows:

$$L_{CFM}(\theta) = E_{x_0 \sim p_0, x_1 \sim p_1} \int_0^1 E_{x(t) \sim p_t} [\|v_{\theta}(x(t), t) - v^*(x(t), t)\|^2] dt$$

This function forces the model v_θ to estimate the optimal conditional vector field $v^*(t|x_0, x_1)$; i.e., the path that connects the two endpoints x_0 and x_1 .

- **B) Flow Matching (FM) Loss Function:** FM or Unconditional Flow Matching learns an unconditional flow that transforms the entire distribution p_0 to p_1 . The FM loss function is defined as the sum of the average of all conditional flows (CFM):

$$L_{FM}(\theta) = E_{x_0 \sim p_0} \int_0^1 E_{x(t) \sim p_t} [\|v_\theta(x(t), t) - v^*(x(t), t)\|^2] dt$$

where v^* is the optimal unconditional flow vector field.

2. Proof of Equivalence (The Equivalence Condition) The relation $\nabla_\theta L_{FM}(\theta) = \nabla_\theta L_{CFM}(\theta)$ holds under the following conditions:

1. **Correct Choice of Conditional Flow:** The chosen conditional flow $v(x(t), t|x_0, x_1)$ must be an optimal local transport flow between $p(x_0)$ and $p(x_1)$, such that the distribution of the total vector field $v(x(t), t)$ is obtained by averaging the conditional vector fields.

$$v^*(x(t), t) = E_{x_0, x_1} [v^*(x(t), t|x_0, x_1)]$$

2. **Use of the CFM Property:** In the main Flow Matching articles, it has been shown that the gradient of the unconditional loss function (FM) is equal to the gradient of the conditional loss function (CFM). This proof is based on the fact that:

- **CFM Calculation is Simple:** Calculating $v^*(x(t), t|x_0, x_1)$ for a specific pair of points (x_0, x_1) (which is called noise reduction in Diffusion) is easy.
- **The Final Goal is the Same:** The goal of CFM is to estimate a vector field that, on average over the data pairs (x_0, x_1) , is the same as the optimal unconditional flow vector field (which is the goal of FM).

Final Reason for the Relation to Hold: In short, the relation $\nabla_\theta L_{FM}(\theta) = \nabla_\theta L_{CFM}(\theta)$ holds because: The gradient of the FM loss function, which is complex and integral, can be calculated and estimated through the gradient of the CFM loss function, which is a simple mean squared error (MSE) loss function. This equivalence allows researchers, instead of calculating the complex integrals required for LFM, to use a simple MSE-based loss function for model training, which is implemented in a practical way as follows:

$$L_{Train}(\theta) = E_{x_0 \sim p_{data}, z \sim p_{noise}, t \sim U(0,1)} [\|v_\theta(x(t), t) - v_{target}(x_0, z, t)\|^2]$$

where $x(t)$ is the sample at time t , and v_{target} is the target vector field (which only connects the two points x_0 and z). This huge computational simplification is what constitutes the core of Flow Matching.

7.1.3 Second Subsection: Proof of the Conditional Vector Field Relation (Conditional Vector Field)

In this section, we use the results of the main Flow Matching article to show the relation related to the conditional vector field $u(t|x_1)$.

Background: Flow Matching and Optimal Transport Flow Matching relies on the concept of Optimal Transport (OT), which seeks to find an optimal mapping (Optimal Map) or a dynamic process (Optimal Flow) that transforms one probability distribution p_0 to another distribution p_1 with minimum possible cost. In Flow Matching, this process is simplified by conditional flows. The following conditional flows are considered:

1. **Conditional Flow $u(t|x_1)$:** This flow defines the optimal path that guides a conditional noise point z to a target data point x_1 .
2. **Noising Process:** This process defines a noise path that takes a data point x_1 to a noise point z (the initial distribution). In Flow Matching models that use Gaussian noise, linear trajectories are often used.

Use of Linear Paths and Gaussian Distribution In the Flow Matching article and related to it, the Rectified Flows article, the optimal transport path that connects two points z and x_1 is considered a linear path (straight line). The linear path between the noise z and the data x_1 at time t is defined as follows:

$$x(t) = \sigma_t z + \mu_t x_1$$

It is assumed that:

- $x(0) = z$ (initial noise distribution, $\mu_0 = 0, \sigma_0 = 1$)
- $x(1) = x_1$ (target data distribution, $\mu_1 = 1, \sigma_1 = 0$)

For linear paths, the conditional flow vector field that takes the point z to x_1 is equal to:

$$v(x(t), t | x_1, z) = \frac{dx(t)}{dt}$$

Since $x(t) = (1 - t)z + tx_1$ (the simplest linear path), then:

$$v(x(t), t | x_1, z) = x_1 - z$$

Proof of the Relation with the Assumption of a Gaussian Linear Path In Theorem 3 of the related articles, this conditional flow is modeled using Gaussian distributions. In the general case (relation 11 in the main article), the distribution of x at time t , conditioned on the target point x_1 , is a Gaussian distribution:

$$x(t) | x_1 \sim N(\mu_t(x_1), \sigma_t^2(x_1)I)$$

where $\mu_t(x_1)$ is the mean and $\sigma_t^2(x_1)$ is the variance. The conditional flow vector field $u(t | x_1)$, which specifies the optimal path for text generation, is obtained through an equation called the Schrödinger Bridge or Optimal Transport. With the assumption of Gaussian property and linear path, Theorem 3 states that the optimal conditional flow vector field follows the following form:

$$u_t(x | x_1) = \frac{\sigma'_t(x_1)}{\sigma_t(x_1)}(x - \mu_t(x_1)) + \mu'_t(x_1)$$

Explanation of Components:

- $u_t(x | x_1)$: The flow vector field (rate of change) at point x and time t , conditioned on the final target point being x_1 .
- $\mu_t(x_1)$ and $\sigma_t(x_1)$: The mean and standard deviation of the distribution of $x(t)$ conditioned on x_1 . In simple linear paths, these are simple functions of t (e.g., $\mu_t = (1 - t)x_0 + tx_1$).
- $\mu'_t(x_1)$ and $\sigma'_t(x_1)$: The derivative of these functions with respect to time t .

Conclusion: This relation shows that in an optimal transport process that uses linear paths and Gaussian distributions to define conditional flows, the conditional vector field (what the model should learn) has a simple analytical structure that is composed of two parts:

1. **Noise Correction part:** $\frac{\sigma'_t(x_1)}{\sigma_t(x_1)}(x - \mu_t(x_1))$, which tries to bring $x(t)$ close to the path's mean $\mu_t(x_1)$.
2. **Transport Velocity part:** $\mu'_t(x_1)$, which shows the net speed of the mean of the path from noise to data.

7.1.4 Third Subsection: The Optimal Transport (OT) Problem

1. Brief Description of the Optimal Transport Problem The Optimal Transport (OT) problem seeks to find the best and most efficient way to transform one mass distribution (e.g., noise distribution) to another mass distribution (e.g., real data distribution), such that the total cost of transport is minimized.

- **Input:** Two probability distributions p_0 (source distribution) and p_1 (target distribution).
- **Output:** An optimal transport map $T(x)$ that takes p_0 to p_1 , or a transport flow $v(x, t)$ (Optimal Flow) that performs this transformation continuously over time.

- **Cost Function:** A cost function $c(x, y)$ that specifies the cost of transporting one unit of mass from point x to point y (usually the Euclidean distance or its square).
- **Goal:** Minimizing the total expected cost:

$$\min E_{x \sim \mu_0} [c(x, T(x))]$$

2. How OT is Used in Flow Matching Flow Matching uses OT to define optimal transport paths, but with a clever difference:

- **OT in the General Case:** OT finding the optimal mapping T for a pair of distributions (μ_0, μ_1) is very computationally heavy.
- **OT in Flow Matching:** The Flow Matching strategy breaks down this difficult problem into the summation of many simpler and local OT problems (Conditional OT).
 - Instead of finding a global optimal flow $v(t)$, the model learns the optimal conditional flows $u(t|x_1)$ (the path between a noise point and a data point).
 - Then, the Flow Matching vector field is obtained by averaging these simple conditional flows. This technique bypasses the direct computational burden of OT.

Aspect	Pros	Cons
Problem Definition	Simple analytical flows: OT allows the optimal flow vector field (v^*) to be defined analytically using linear paths that have a simple structure (as we saw in the second subsection).	Heavy direct computation: Calculating the unconditional optimal flow vector field (the main goal of OT) is very complex and practically impossible in high dimensions, which is solved by CFM.
Model Learning	Training stability: The CFM loss function, which originates from OT, is a very simple and stable mean squared error (MSE) loss function that makes the training of the neural network reliable.	Dependence on the path: The quality of the model depends on the assumption that the chosen linear path is a reasonable approximation for the optimal flow. If the data requires a more complex path, the performance will decrease.
Sampling	Deterministic generation: OT flows, unlike random Diffusion, define a deterministic transformation path, which allows for faster sampling by solving ODEs.	Need to solve ODE: In the sampling stage, to calculate the path, ordinary differential equations (ODE) must be solved with high accuracy, which may have a higher cost compared to simpler generative networks.

3. Advantages and Disadvantages of Using OT in Flow Matching

Overall Review of Optimal Transport

- **1.1. Statement of the Problem:** In the optimal transport (OT) problem, the goal is to relate two distributions (e.g., α and β) in a space R^d (or a manifold) with the "least cost".
 - The distribution α can be considered as the "source distribution" and β as the "target distribution".
 - The "cost" is usually a function of the distance, such as $\|x - y\|$ or $\|x - y\|^2$, which states how much it costs to move a unit of mass from point x in α to point y in β .

- **1.2. Mathematical Form:** One of the functions that is often used in OT is:

$$\min_T E_{x \sim \alpha}[c(x, T(x))], \quad \text{if } T_*\alpha = \beta$$

where:

- $T : R^d \rightarrow R^d$ is the transport map.
- $T_*\alpha = \beta$ means that if we sample from α and then pass it through T , we will eventually reach β .

- **1.3. Key Point:** When the cost function is the squared distance ($\|x - y\|^2$), in some cases (e.g., simple Gaussian to Gaussian transformation), the OT solution is unique and usually linear (or a straight line) between the corresponding points. This point is very important because in recent works of Flow Matching, this linearity can be used for simplifying the learning.

2. Use of OT in Flow Matching Models Flow Matching (FM) models essentially want to create a time-dependent probabilistic path that takes a simple noise distribution (like $N(0, I)$) at $t = 0$ to the data distribution at $t = 1$. In popular articles, this path is often implemented with the idea of diffusion; i.e., noise is gradually transformed into data. But one of the new suggestions is to use the optimal transport path:

1. **Design of Conditional Path (Conditional OT):** For each real sample x_1 , one can define a path $\{p_t(x|x_1)\}$ such that at $t = 0$, it is a simple noise, and at $t = 1$, it is a small Gaussian around x_1 . With the OT approach, this change can be a straight line in the feature space (i.e., each particle (x) moves directly towards (x_1)).
2. **Simpler Vector Field:** If the map ψ_t between the two Gaussian distributions (noise and data) is constructed, in practice, the speed of movement of the particles (the vector field) comes out in a relatively simple form; the particles move with a constant or straight-line speed towards their destination. In FM, it is enough for us to have that vector field in a closed-form (in the CFM equation) and have the neural network approximate it with v_θ .
3. **Connection with Flow Matching:** In Flow Matching, the final goal is for the vector field $\{v_\theta\}$ to be learned so that the probabilistic output path takes the data from noise to data. If we choose the OT path, learning this vector field becomes easier; because the paths are usually straight and have less fluctuation.

3. Advantages of Using OT in Flow Matching

1. **Linear Paths (Easier Learning):** The OT path in many Gaussian cases is a linear or near-linear path, which makes its learning for the model simpler. In diffusion, sometimes we have a very winding path that the network must learn (over a long time).
2. **Faster Training:** Because the vector field (Velocity Field) is simple, the gradients and the learning phase reach the result faster. The article shows that the OT-Based model often converges sooner.
3. **Efficient Sampling:** When the model is trained, to generate a sample (Sampling), the ODE equation must be solved from $t = 0$ to $t = 1$. If the path is simple, the ODE solver can reach an acceptable accuracy and quality with fewer steps (fewer NFE). Because it is not necessary to follow unnecessary rotations or deviations.
4. **Quality or Better Generalization:** In some tests, metrics like the final log-likelihood or FID (image quality) have been reported to be better than the diffusion method. The probable reason: the model does not learn "extra" vectors and directly focuses on the main part of the path.

4. Disadvantages or Limitations of OT in Flow Matching

1. **In Very Complex Distributions, OT May Not Be Ideal:** If the data distribution is very multi-modal or non-convex, the OT path may be harder for the network to learn or may not be able to create some "noise-like" effects that might be useful in diffusion.

2. **Need for Conditional Path Design:** In this method, we must consider a small Gaussian around each x_1 with σ_{min} ; if σ_{min} is not chosen well, the sample diversity may be low, or the learning process may be disrupted.
3. **Closed-form is not always available:** When the two distributions (noise and data) are very complex, finding the OT map explicitly (like the linear one in the Gaussian case) may not be feasible or may be very complex. The advantage of simplicity is lost in such cases.

Summary:

- Optimal Transport is a classic method for transforming between two distributions with "minimum cost".
- In Flow Matching, using OT allows us to create linear or relatively simple paths, which, while preserving desirable properties, speeds up the learning and sample generation.
- At the same time, if the distribution is very complex or the conditional path design is not appropriate, OT may not show its advantage or may even be troublesome.

As a result, OT in Flow Matching is an attractive and efficient option; especially if the initial and target distributions (noise and data) are simple or close to the Gaussian case and we can find its closed form.

References

- [1] PaliGemma: A versatile 3B VLM for transfer
- [2] Sigmoid Loss for Language Image Pre-Training
- [3] LoRA: Low-Rank Adaptation of Large Language Models
- [4] Consistency-guided Prompt Learning for Vision-Language Models
- [5] Flow Matching for Generative Modeling